Friedrich-Alexander-Universität
Technische Fakultät

Design and Evaluation of Partially Reconfigurable CNN Architectures on FPGAs

Bachelor Thesis Presentation

Maxim Balajan

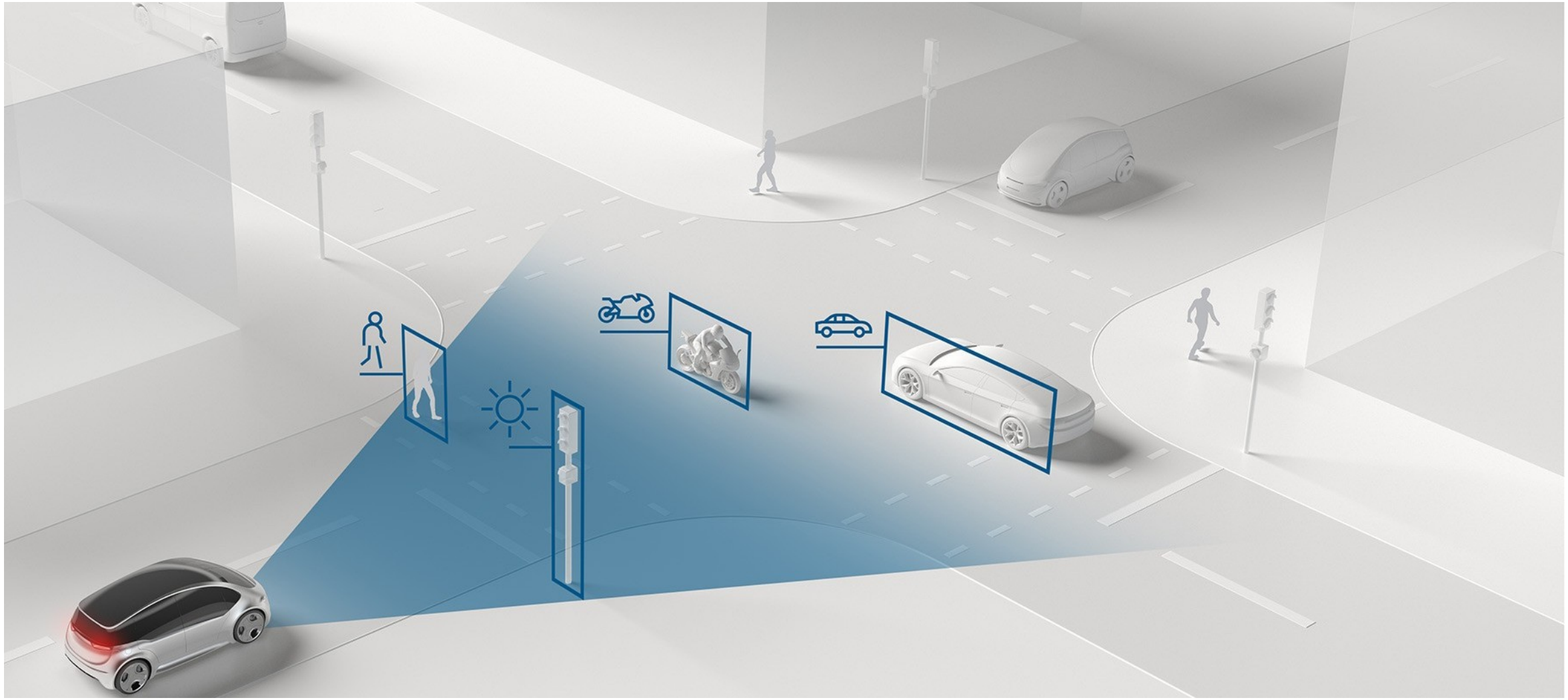
Matrikelnummer: 23228101
StudOn-Kennung: hi46pubi
E-Mail: maxim.balajan@fau.de

January 13, 2026

1. Introduction
2. Architecture Overview
3. Implementation
4. Evaluation
5. Conclusion
6. References

Introduction

Letting Automotives see the World!



Source: https://www.bosch-mobility.com/de/mobility-themen/automatisiertes-fahren-sense-think-act/sense/

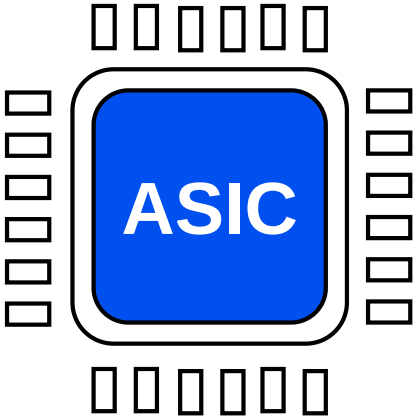
Letting Automotives see the World!

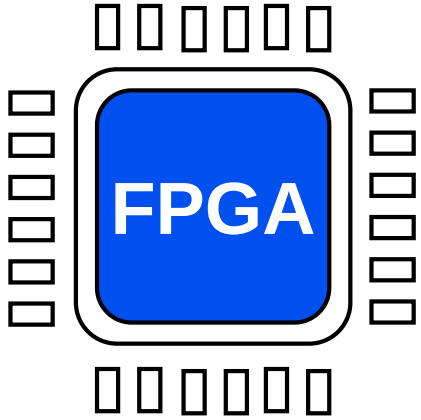


Source: https://www.bosch-mobility.com/de/mobility-themen/automatisiertes-fahren-sense-think-act/sense/

Computer Vision Acceleration

ASIC vs. FPGA

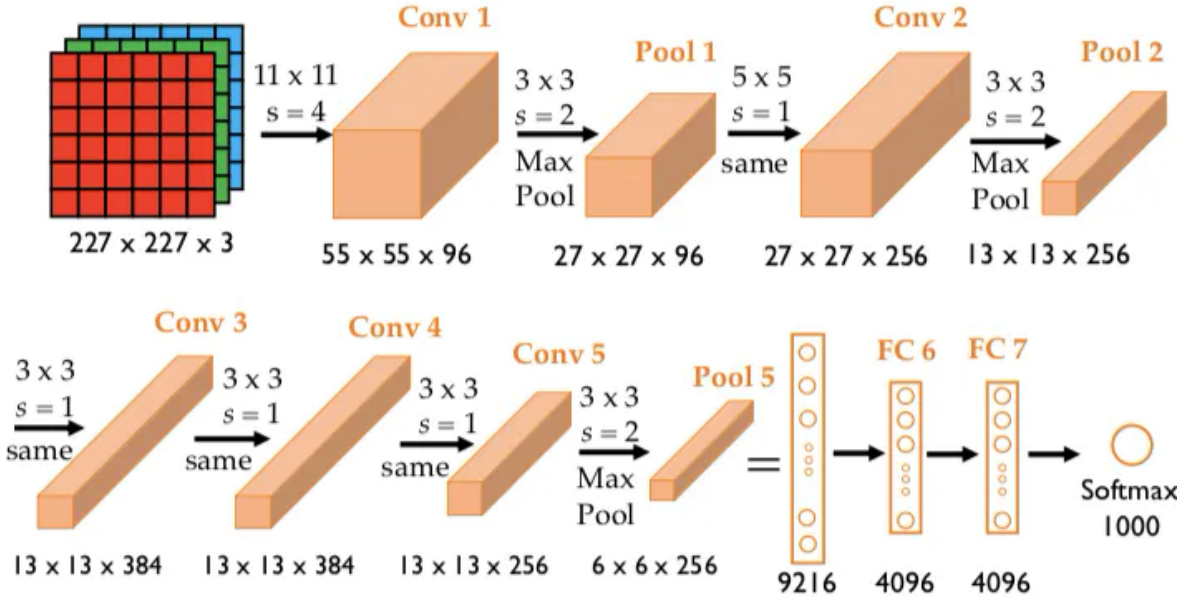




+ Economically cheaper when mass-produced
+ Overall lower energy consumption
– Algorithms can't be updated in hardware
– Designs can't be adapted dynamically

– Economically more expensive when mass-produced
– Overall higher energy consumption
+ Algorithms can be updated in hardware
+ Designs can be adapted dynamically

⇒ ***Dynamic Partial Reconfiguration (DPR)*** of FPGAs as a possibility to stay flexible in the future!



A. Krizhevsky, I. Sutskever, and G. E. Hinton. "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Ed. by F. Pereira, C. Burges, L. Bottou, and K. Weinberger. Vol. 25. Curran Associates, Inc., 2012

Goals:

- Generic architecture for CNN convolution/pooling layers

⇒ **DPR** potential on layer granularity

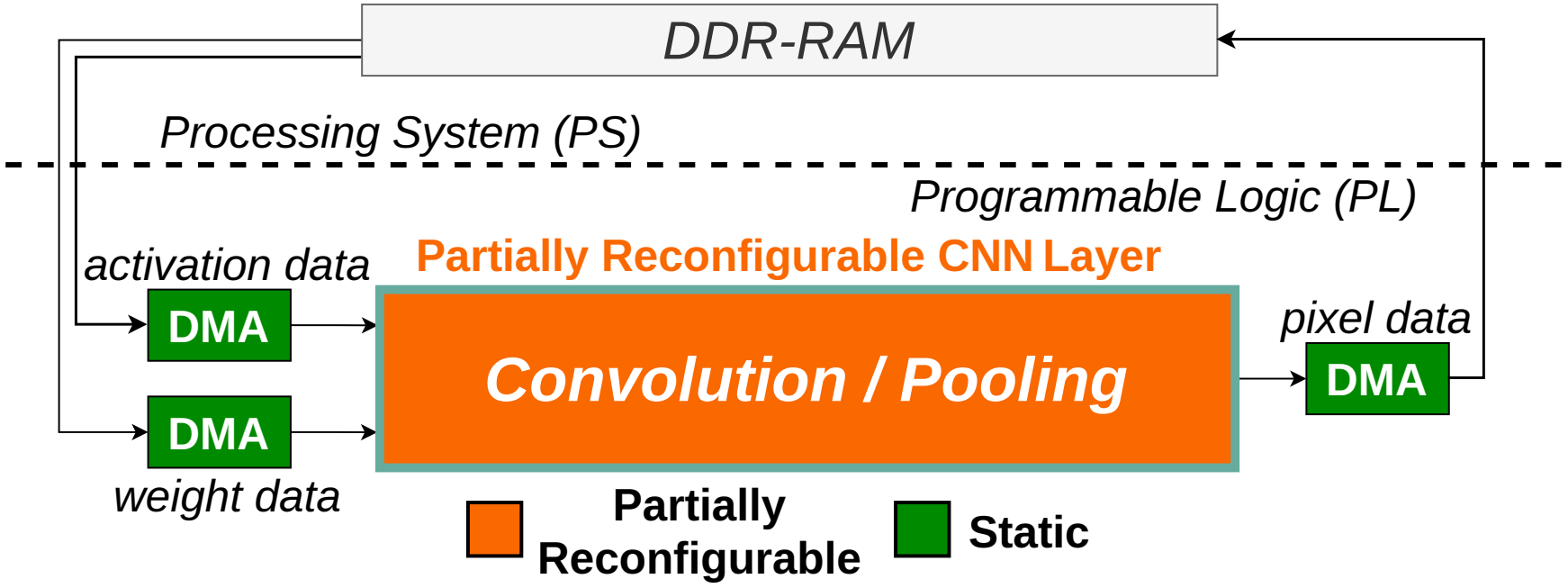
- Architecture for quantized networks (*bit_width* = 8)
- Evaluation of performance and hardware scalability

⇒ Evaluation of reconfiguration times outside of scope

1. Introduction
2. Architecture Overview
3. Implementation
4. Evaluation
5. Conclusion
6. References

Architecture Overview

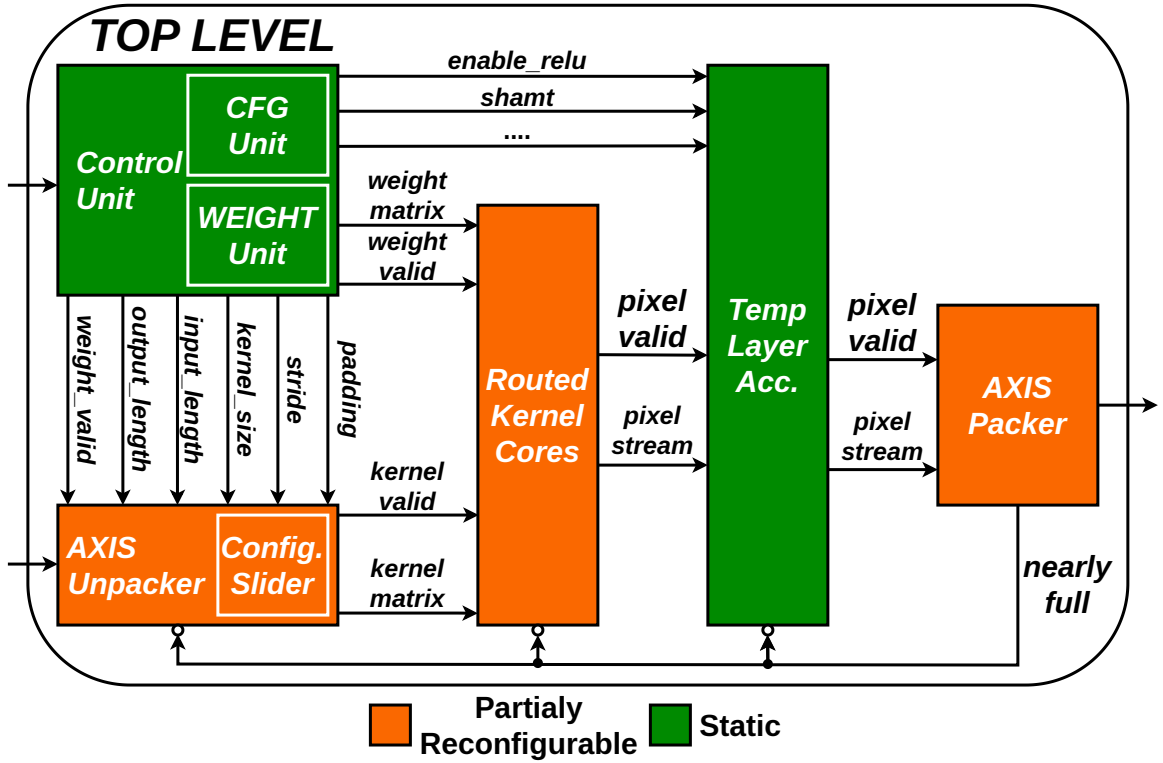
Simplified Hardware-Software-Co-Design

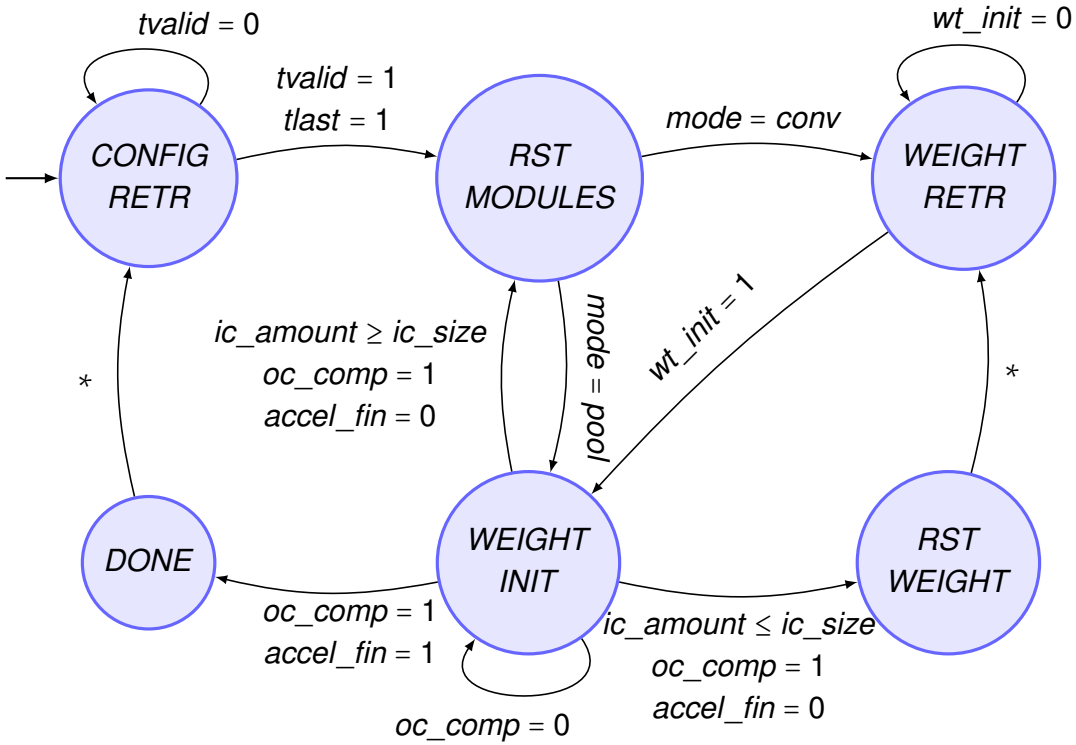


- *Processing System (PS)* sends data over *AXI* (*bus_width* = 128 *Bits*) to the *Programmable Logic (PL)*
- Data is being processed by the accelerator placed on the **PL**
- New layer data is streamed from **PL** back to **PS**

Accelerator Design

Example Computation [IN-CH = 16, C-CH = 8, OUT-CH = 1]

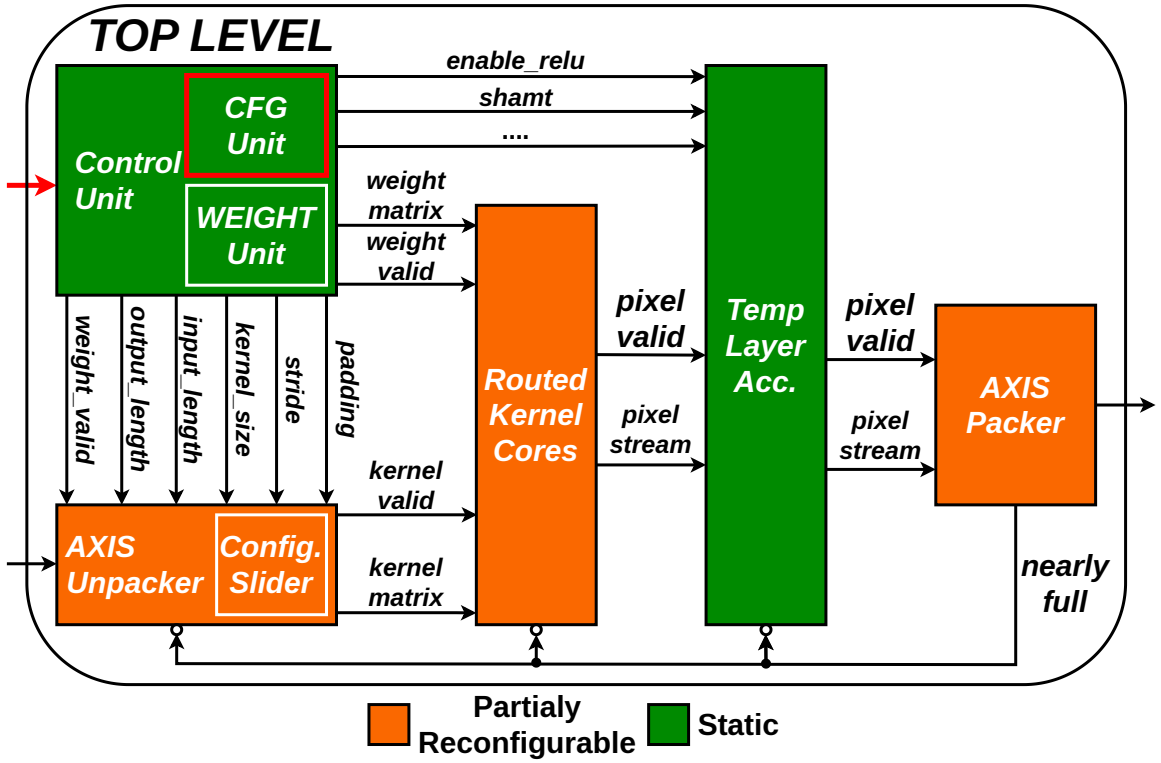


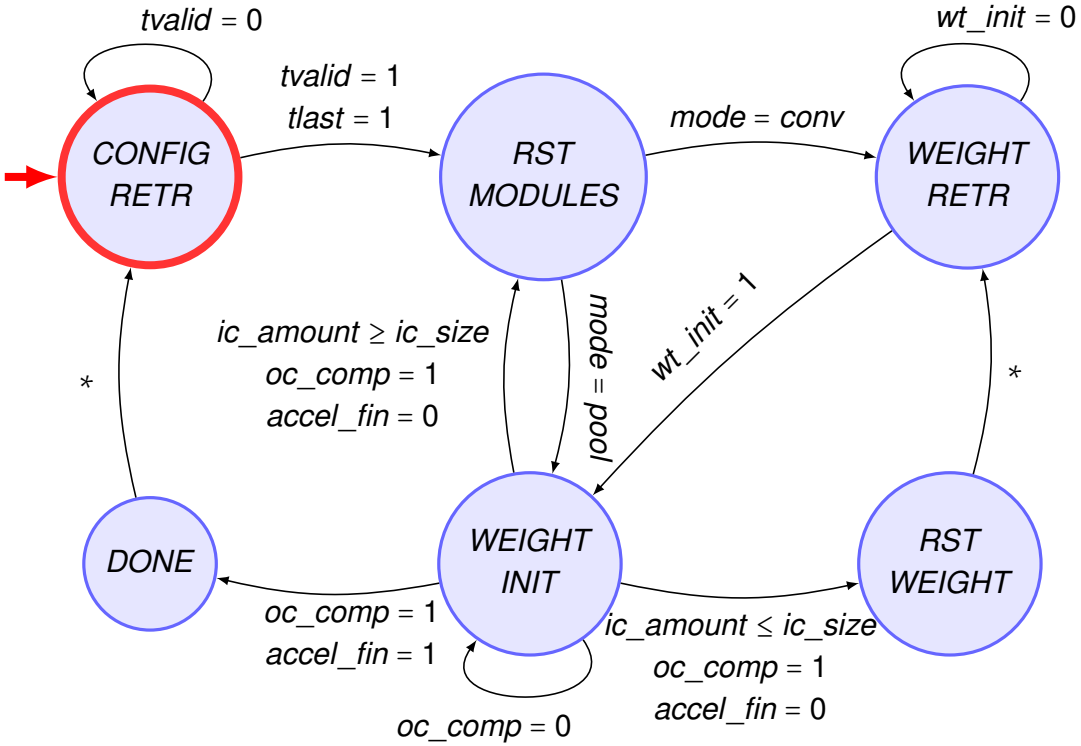


1. **Convolution Phase**: Computation over *two* iterations

Accelerator Design

Example Computation [IN-CH = 16, C-CH = 8, OUT-CH = 1]

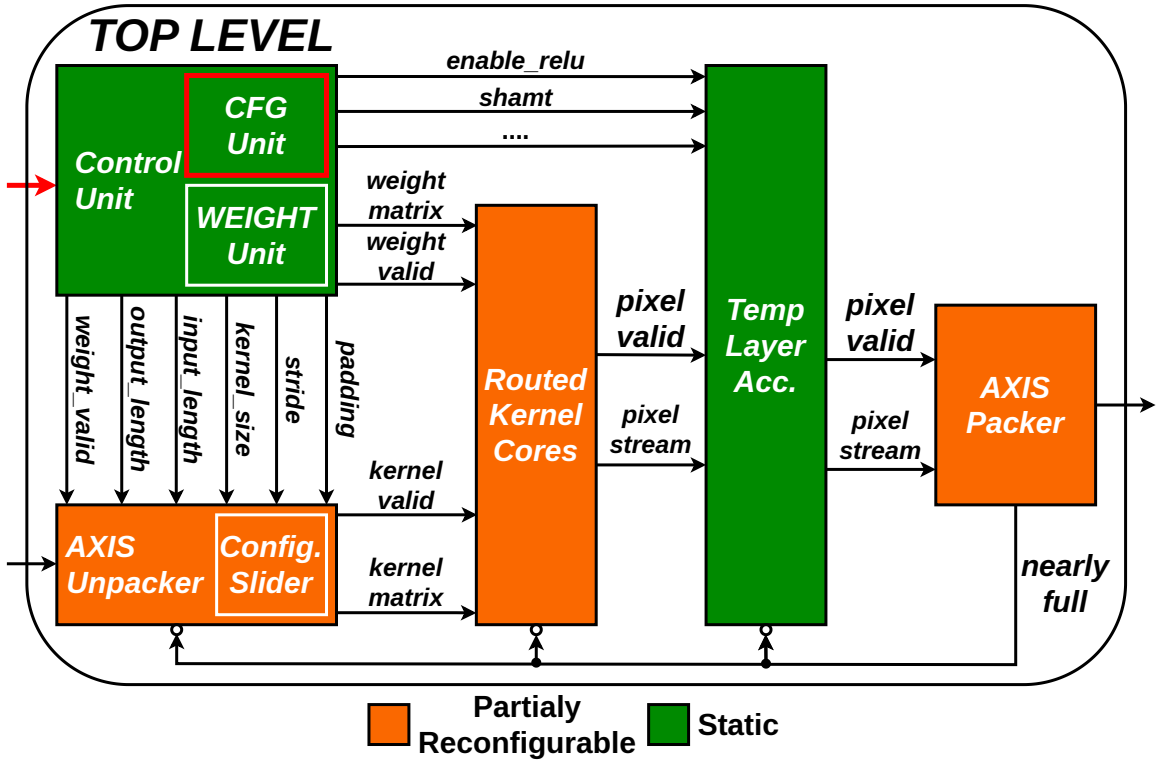


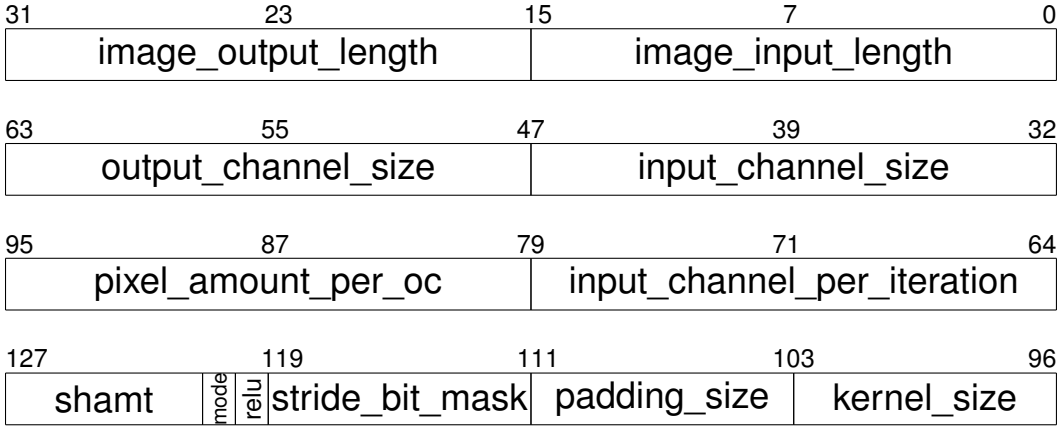


2. *Retrieval* of various configuration data

Accelerator Design

Example Computation [IN-CH = 16, C-CH = 8, OUT-CH = 1]

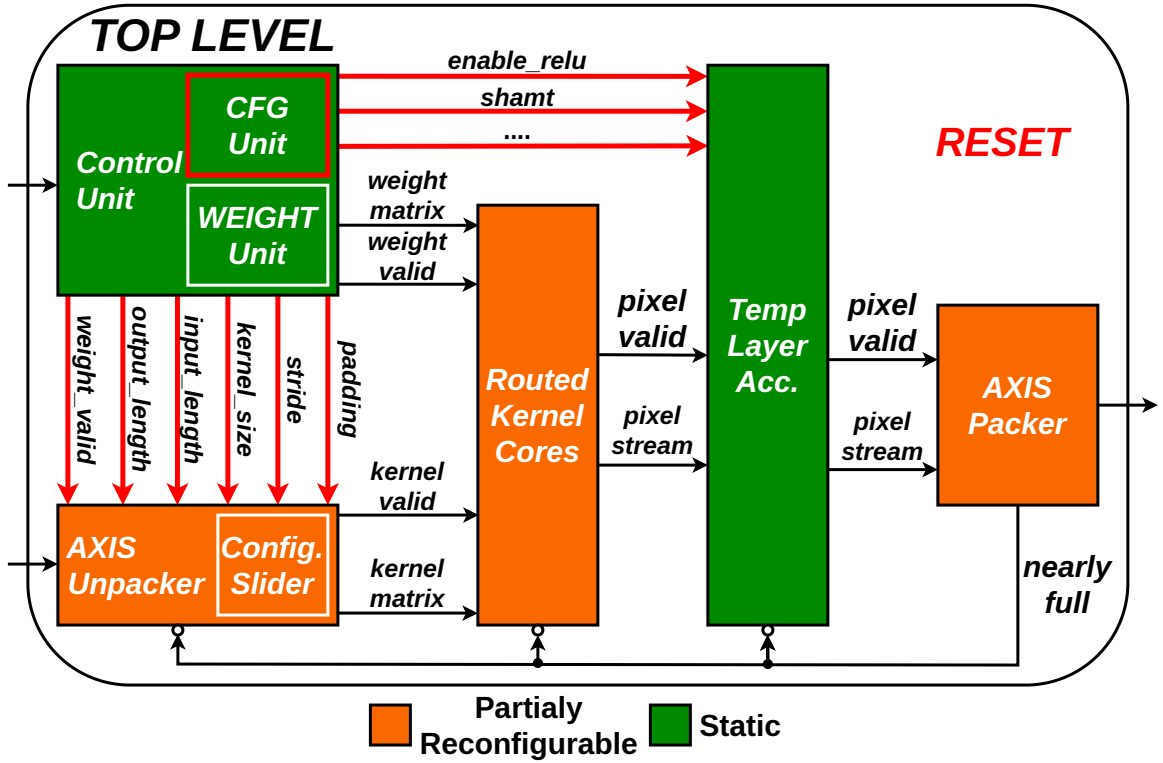


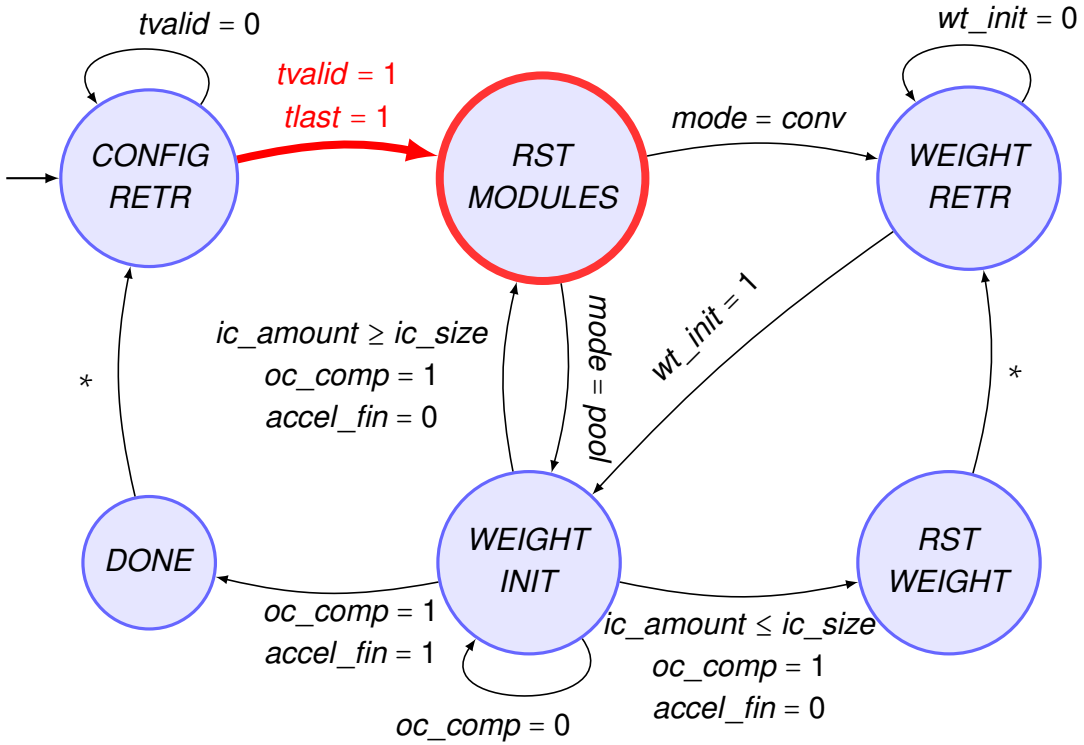


3. *Retrieval* of various configuration data

Accelerator Design

Example Computation [IN-CH = 16, C-CH = 8, OUT-CH = 1]

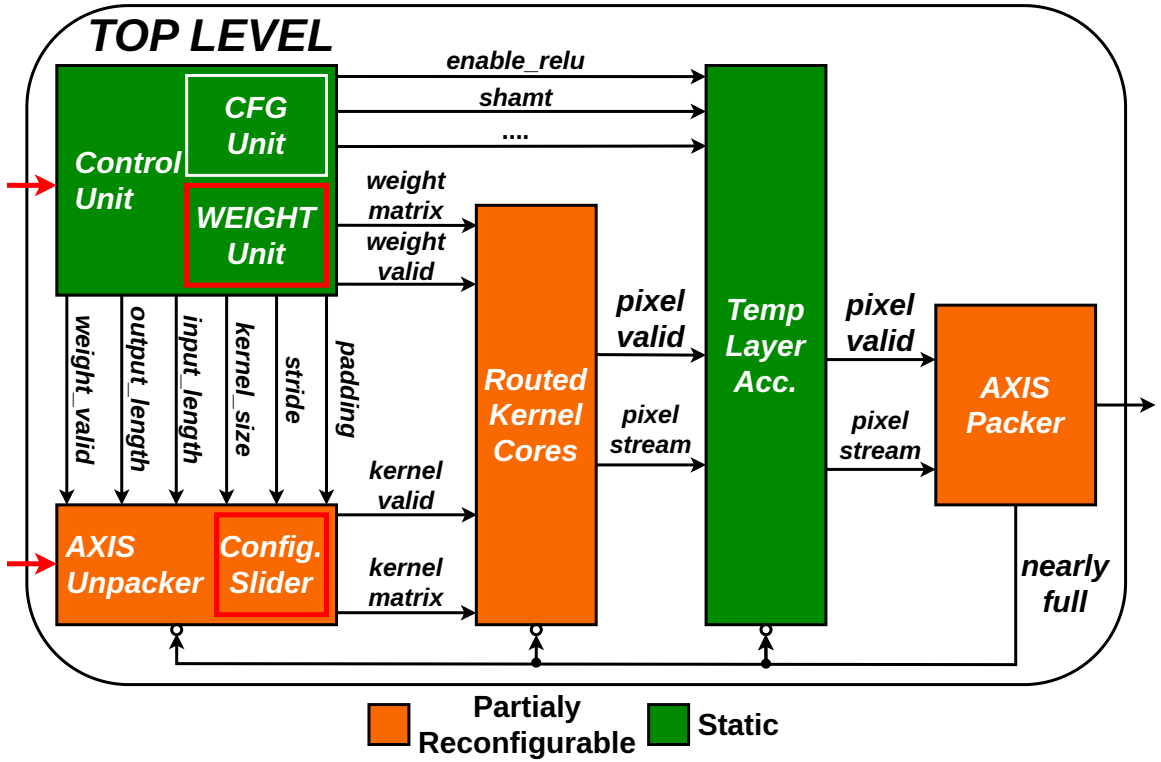


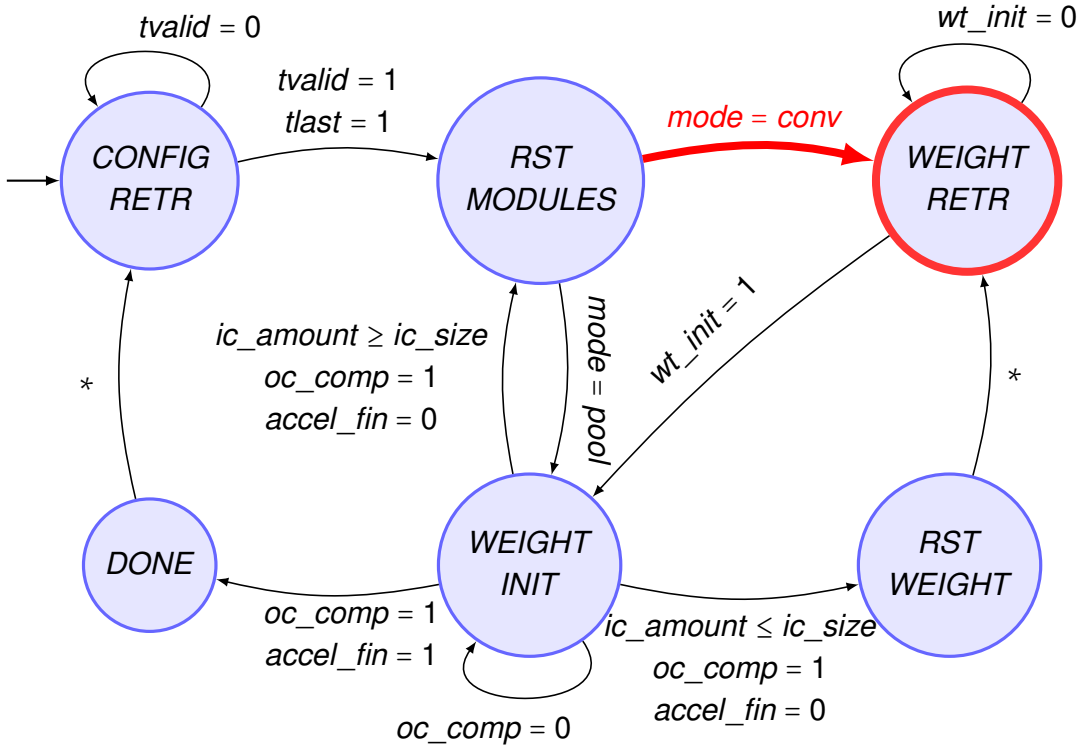


4. *Programming* of device using configuration string

Accelerator Design

Example Computation [IN-CH = 16, C-CH = 8, OUT-CH = 1]

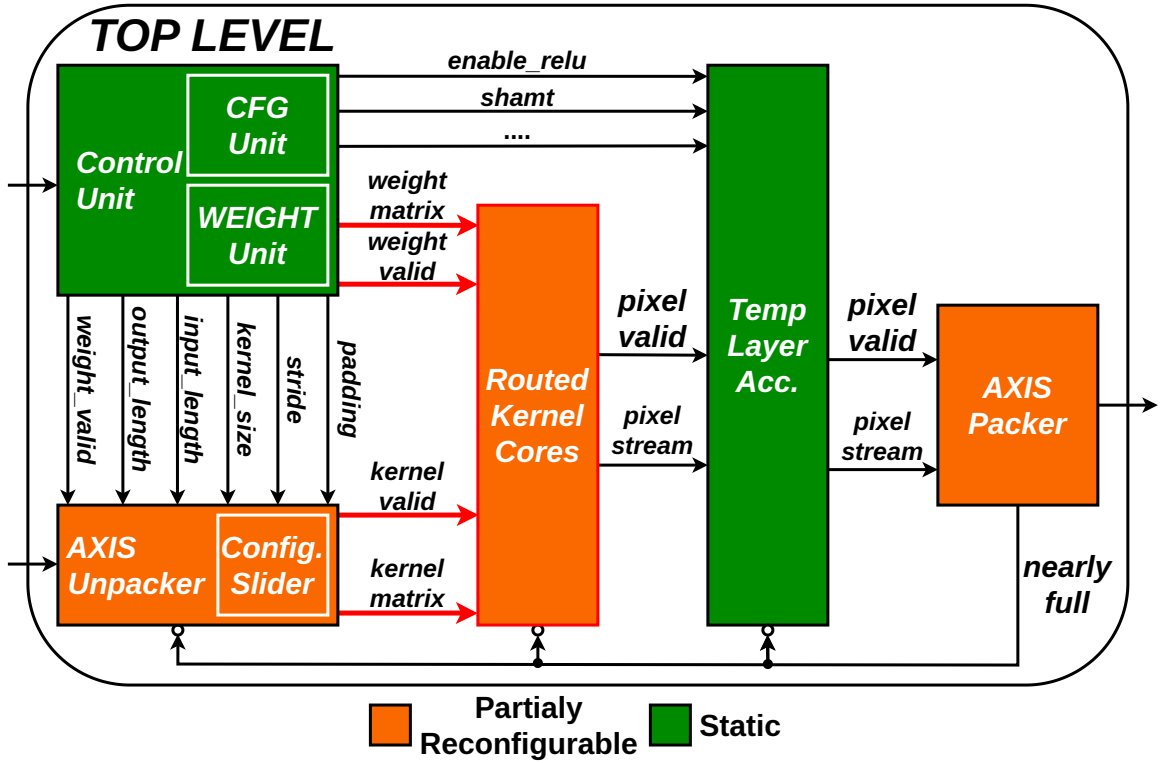


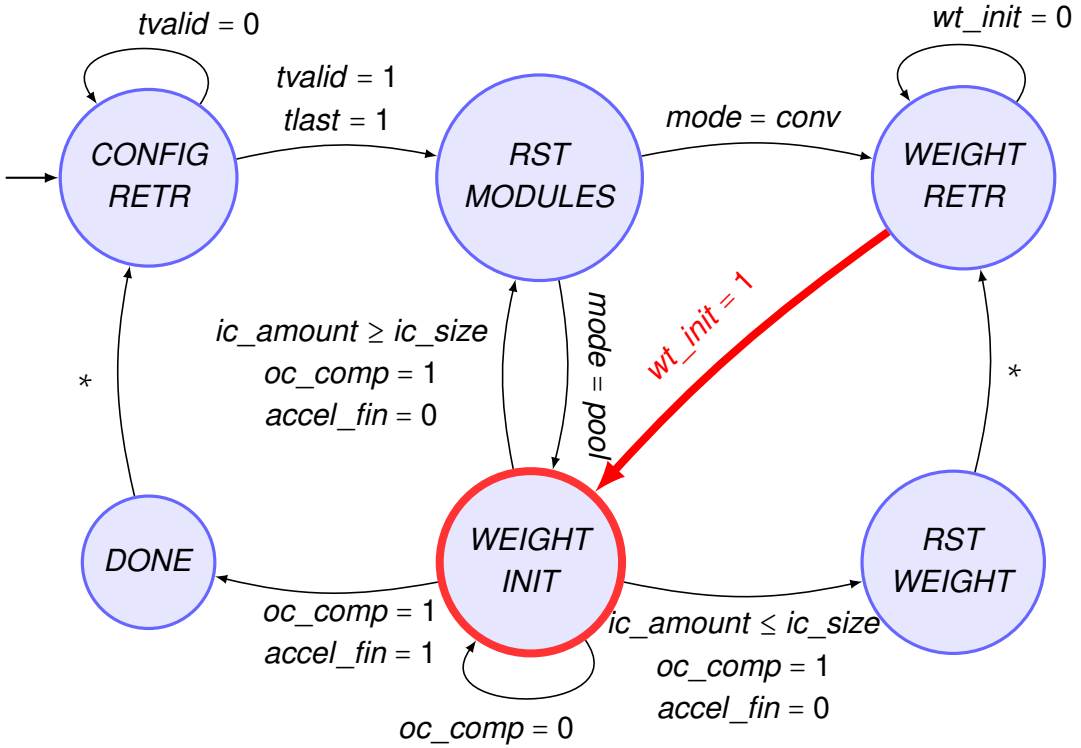


5. Starting retrieval of **weight** and **pixel** data

Accelerator Design

Example Computation [IN-CH = 16, C-CH = 8, OUT-CH = 1]

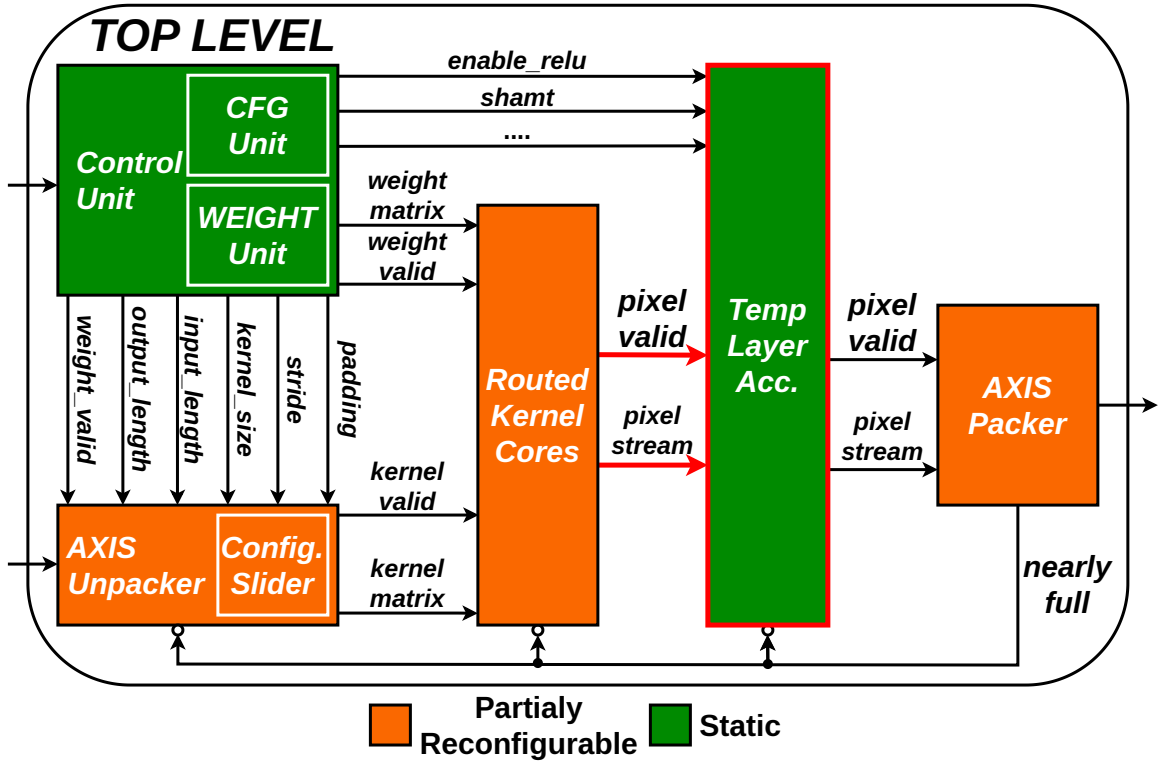


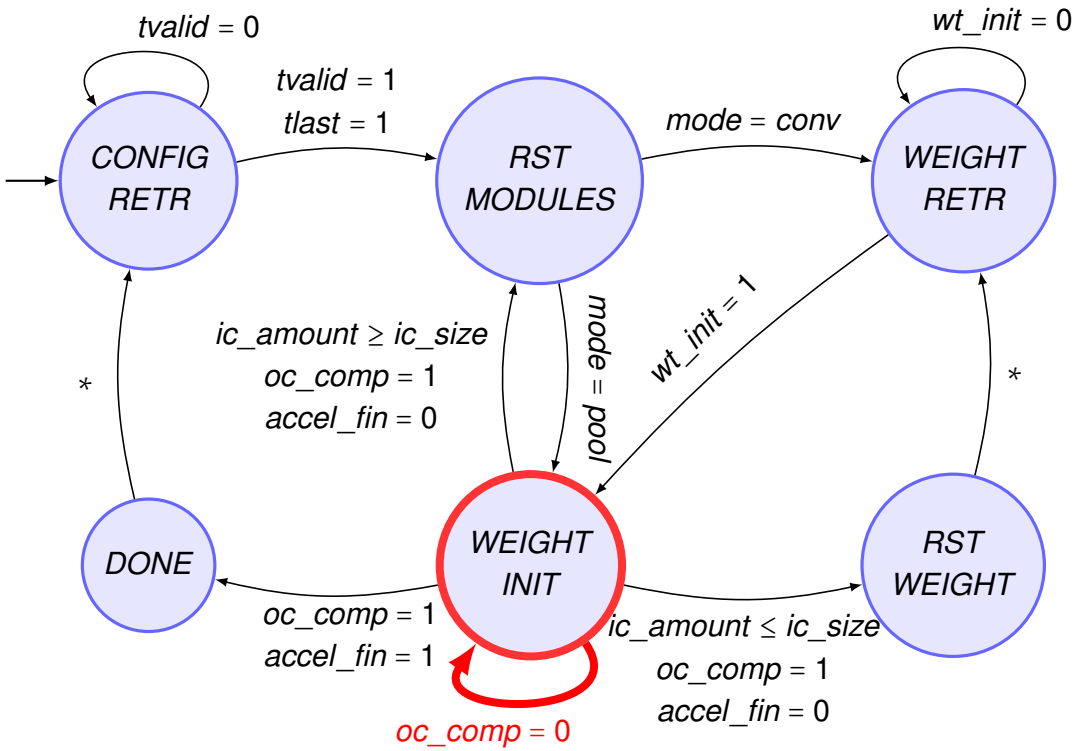


6. Weight and Activation kernels send through adder trees

Accelerator Design

Example Computation [IN-CH = 16, C-CH = 8, OUT-CH = 1]

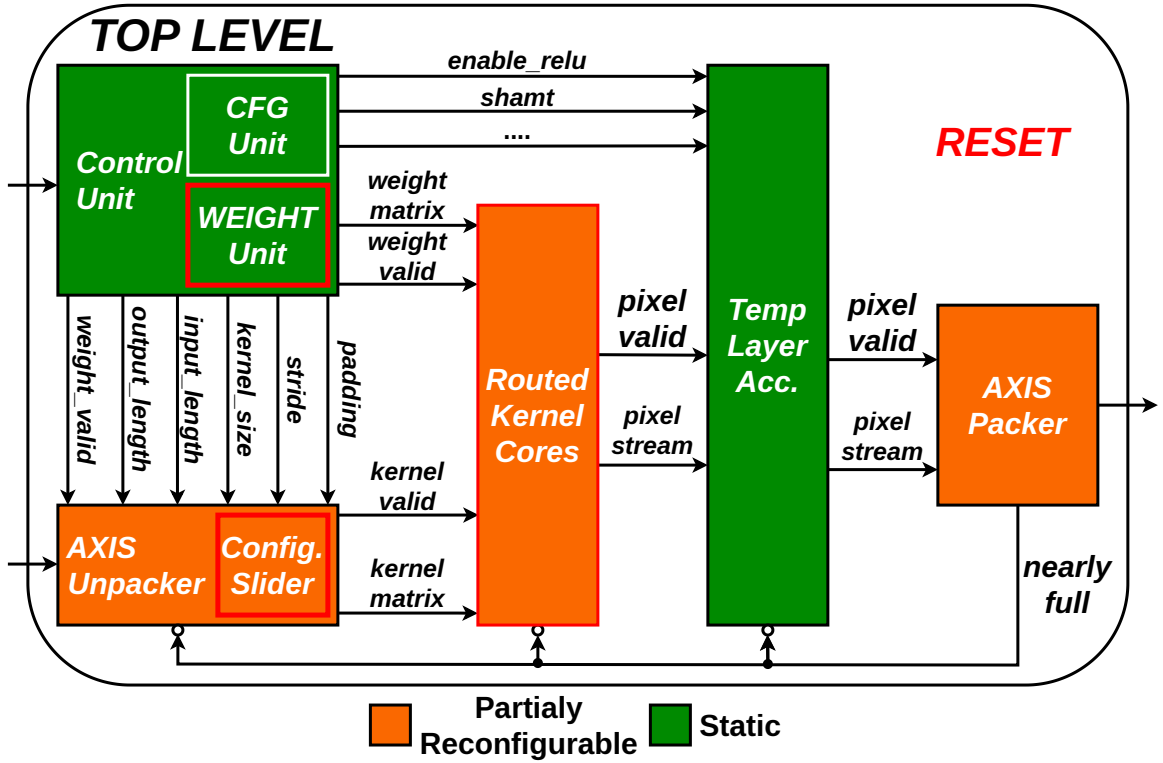


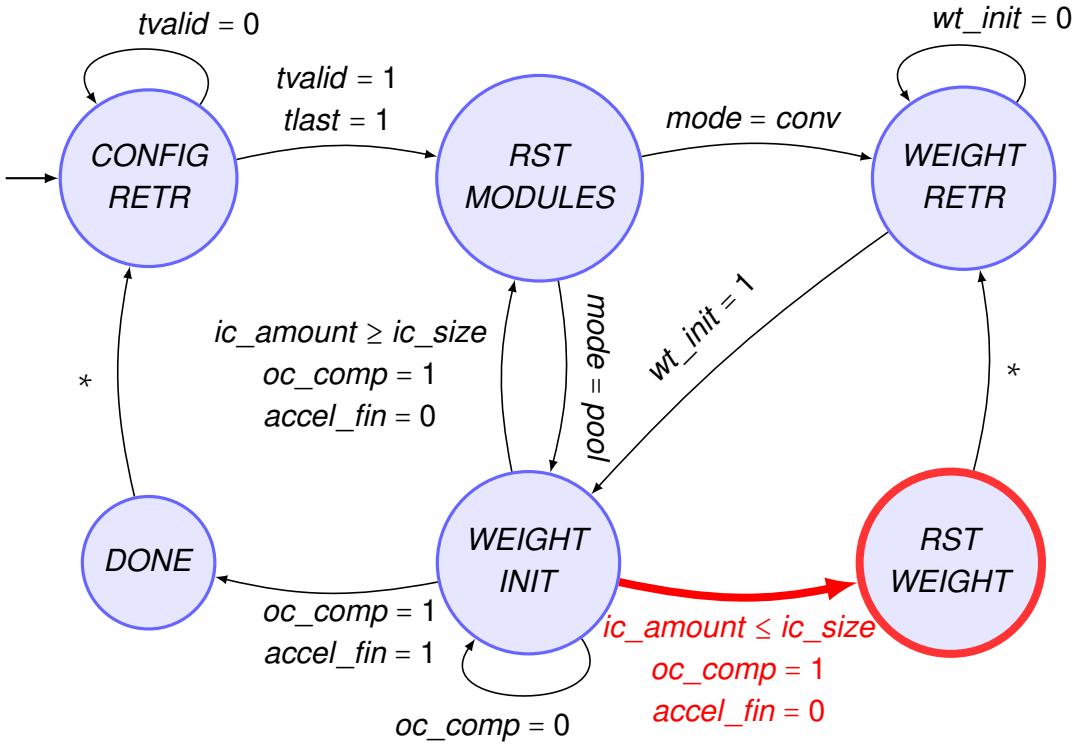


7. Temporary output channel stored inside accumulator

Accelerator Design

Example Computation [IN-CH = 16, C-CH = 8, OUT-CH = 1]

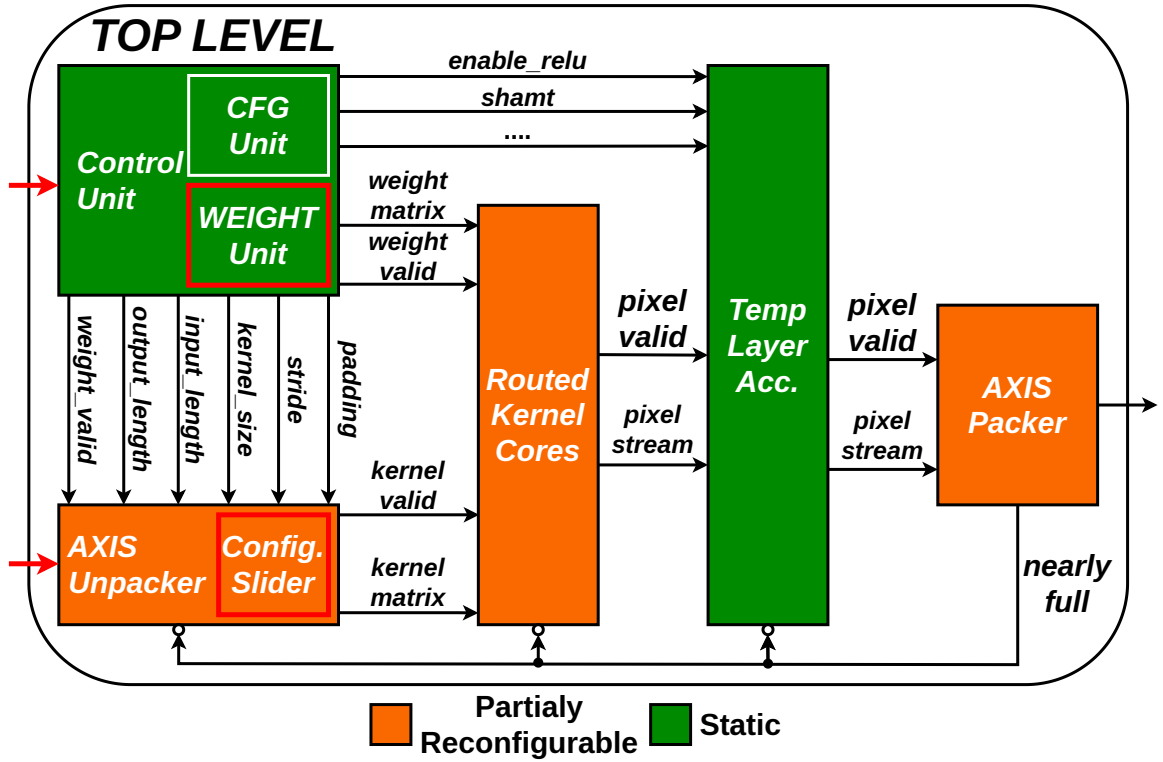


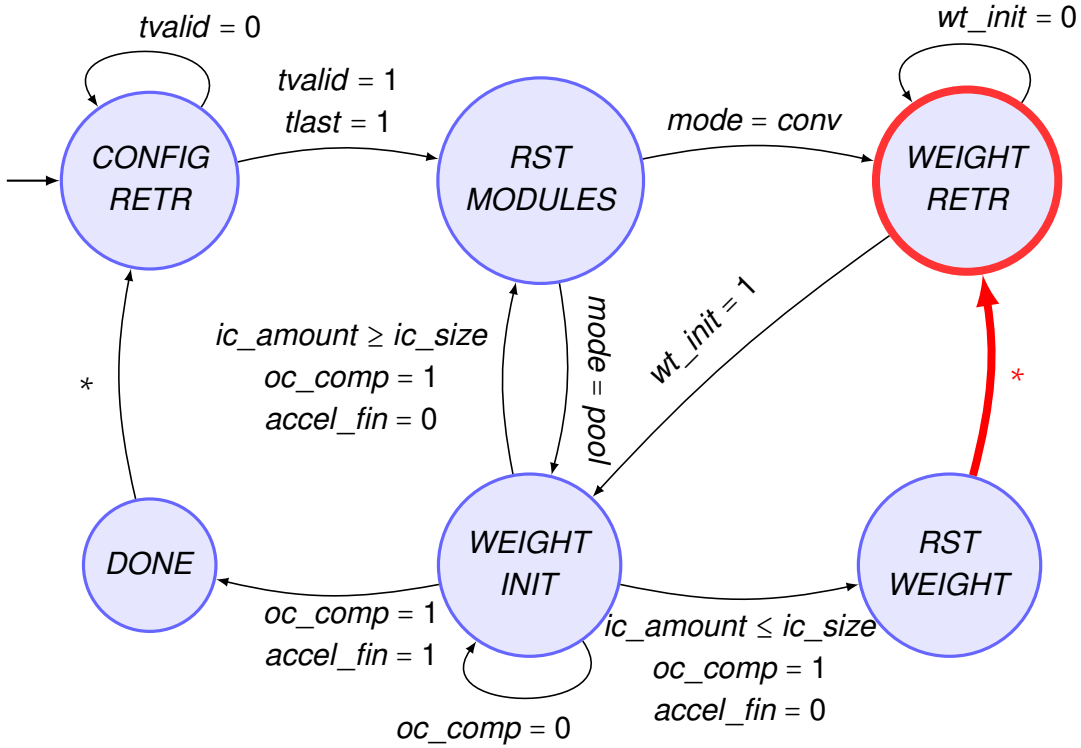


8. Repeat steps for *second* iteration

Accelerator Design

Example Computation [IN-CH = 16, C-CH = 8, OUT-CH = 1]

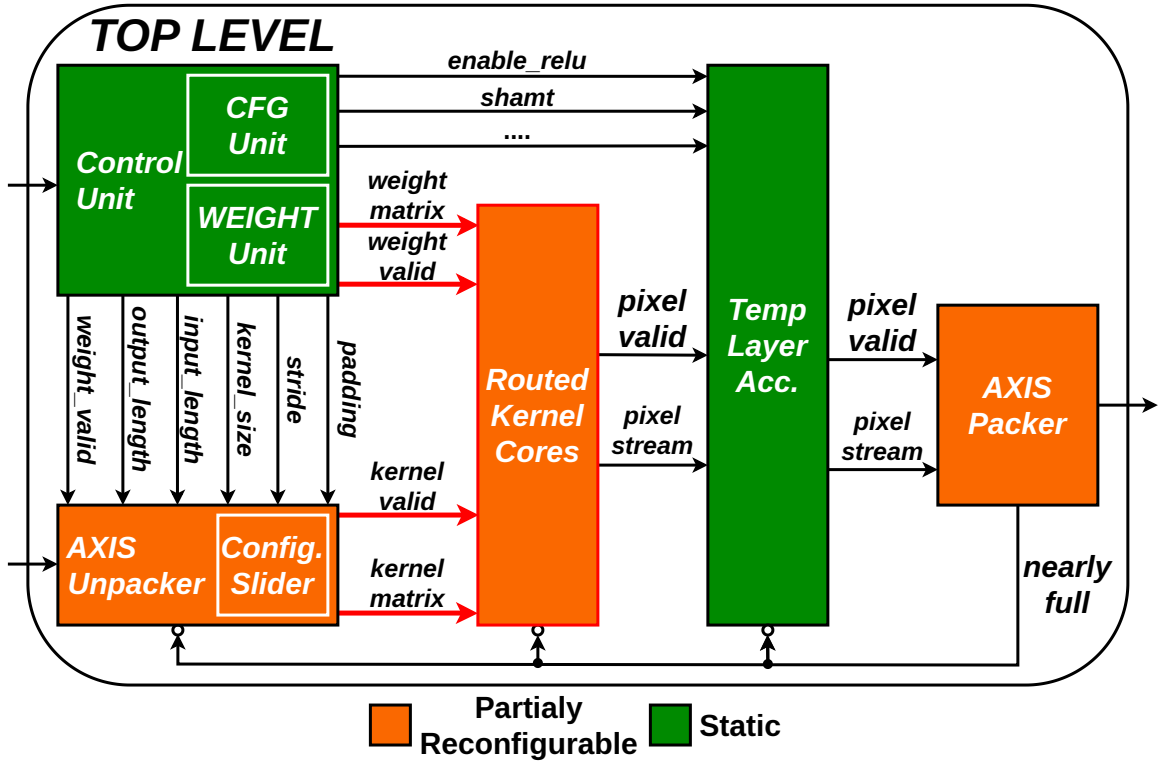


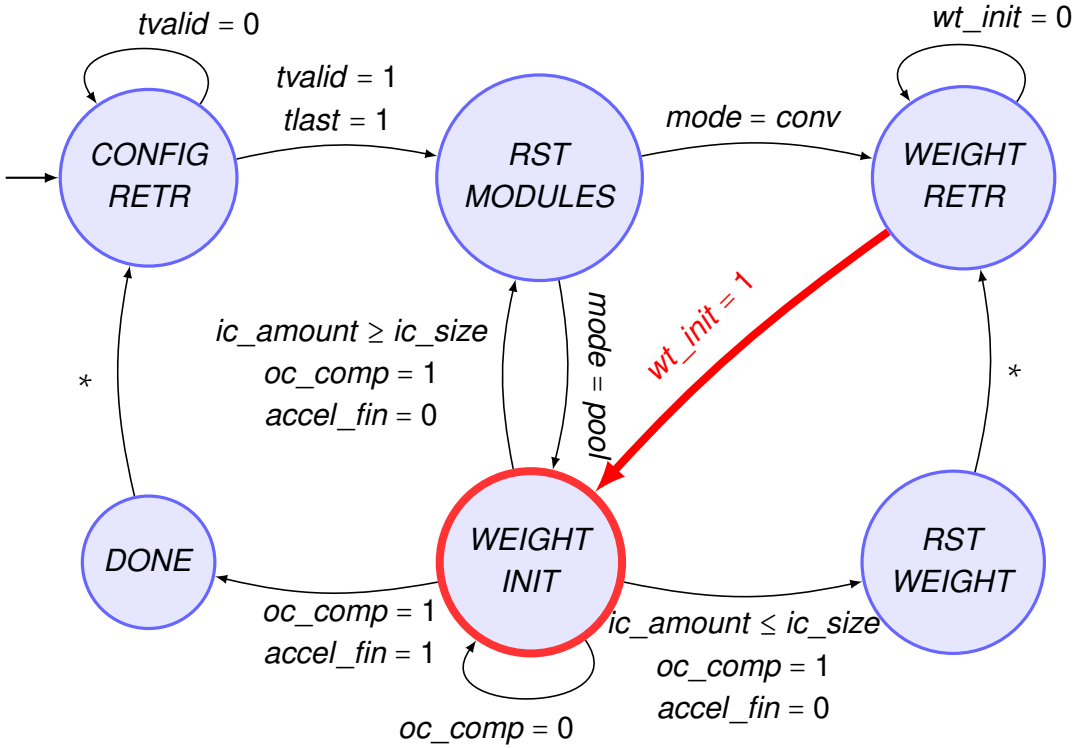


9. Repeat steps for *second* iteration

Accelerator Design

Example Computation [IN-CH = 16, C-CH = 8, OUT-CH = 1]

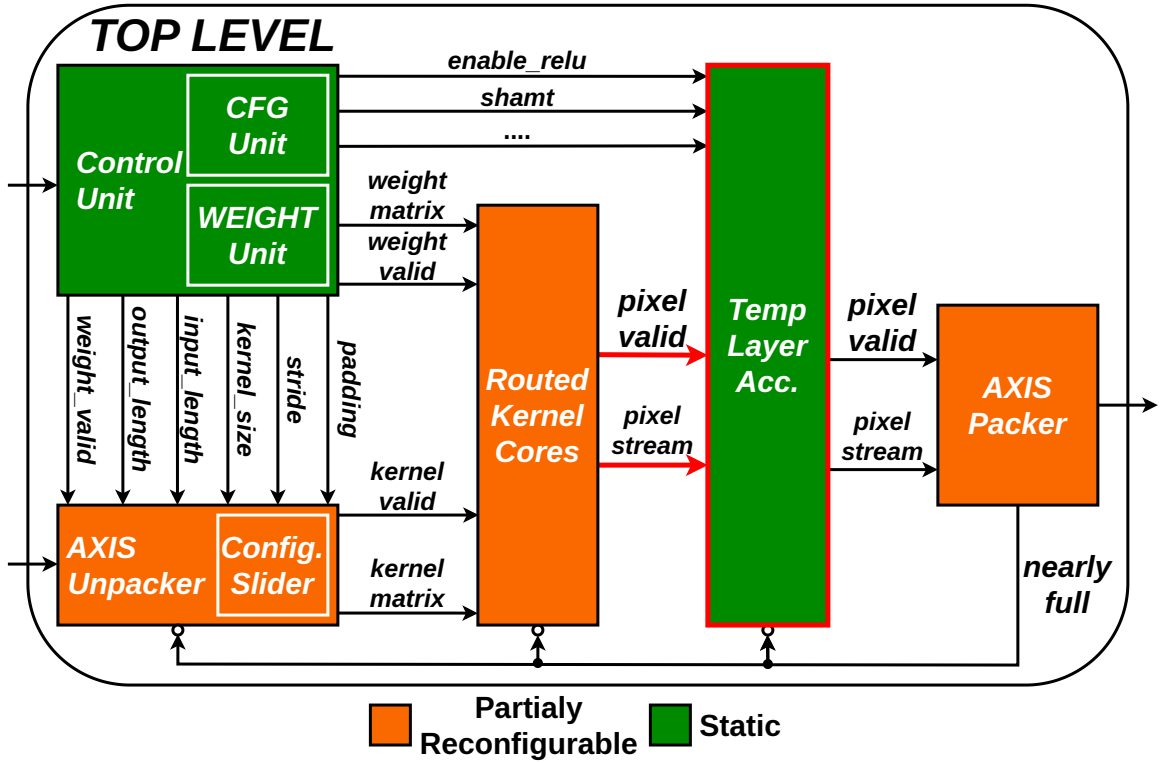


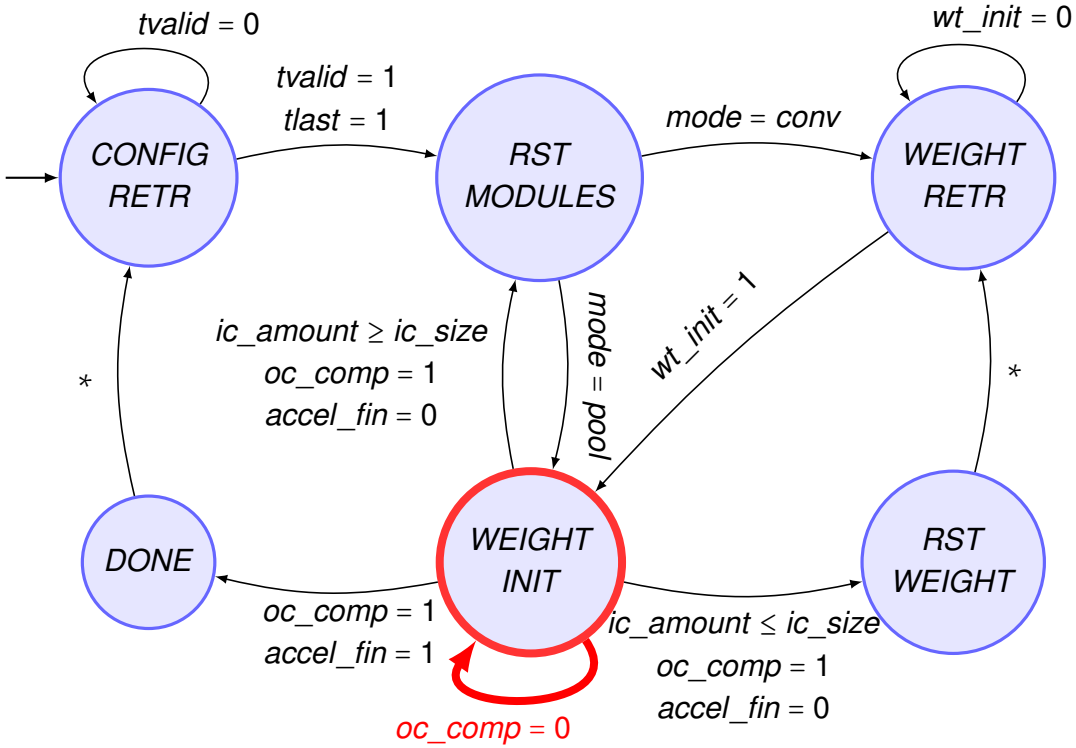


10. Repeat steps for *second* iteration

Accelerator Design

Example Computation [IN-CH = 16, C-CH = 8, OUT-CH = 1]

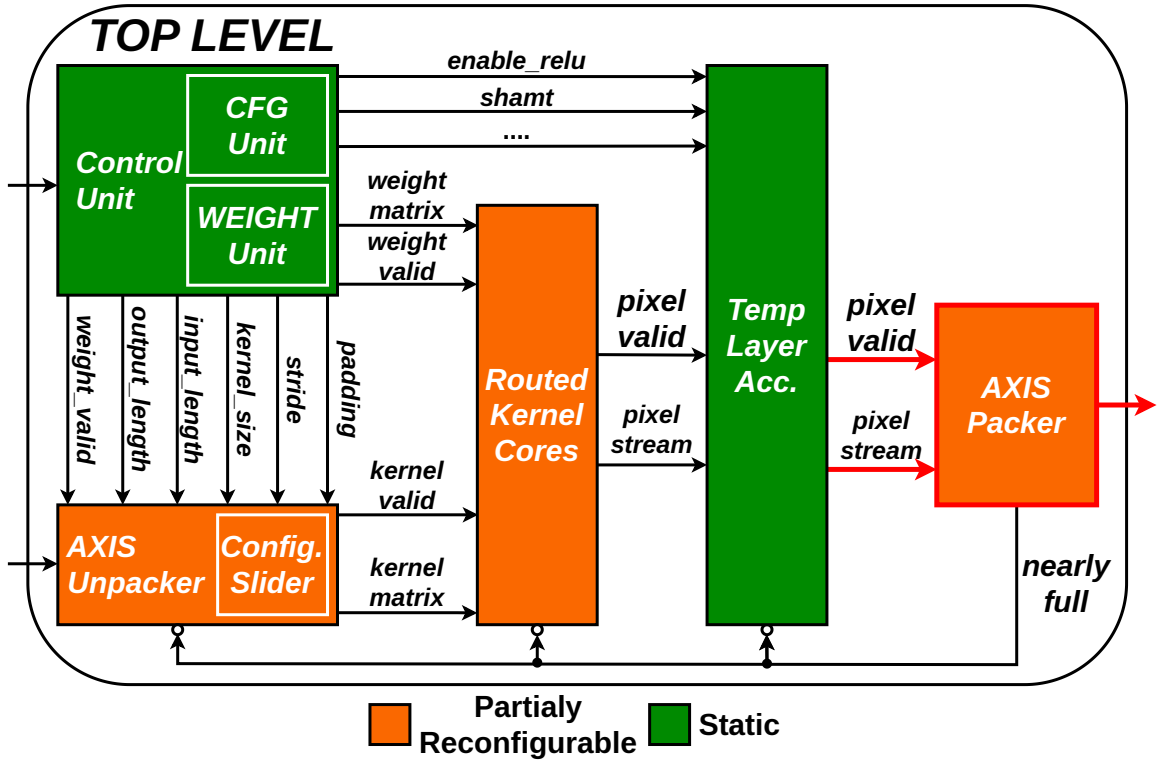


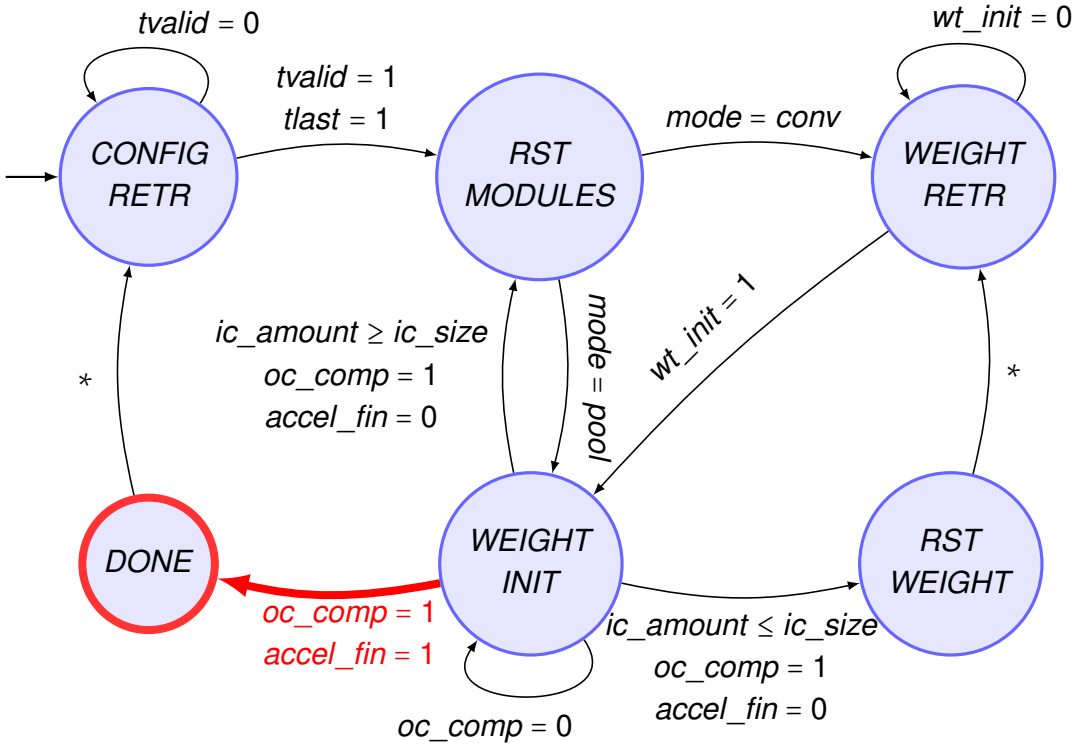


11. Repeat steps for *second* iteration

Accelerator Design

Example Computation [IN-CH = 16, C-CH = 8, OUT-CH = 1]

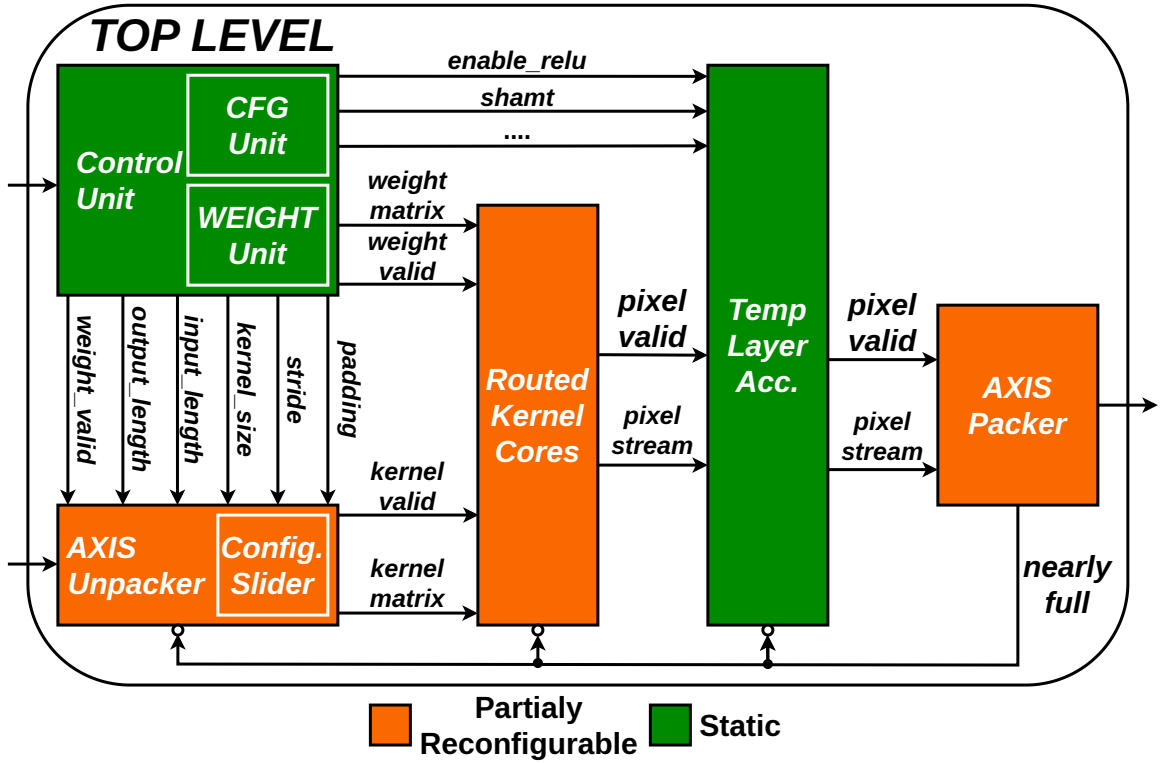


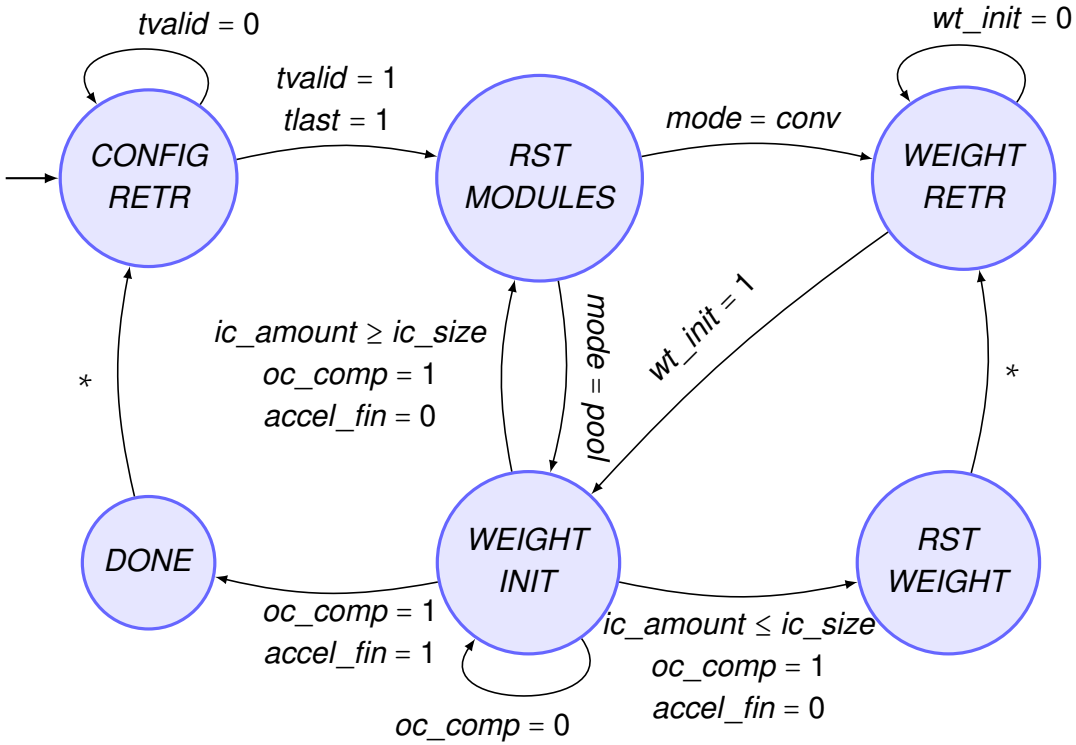


12. Sending of new channel data via *AXI-Stream* to **PS**

Accelerator Design

Example Computation [IN-CH = 16, C-CH = 8, OUT-CH = 1]

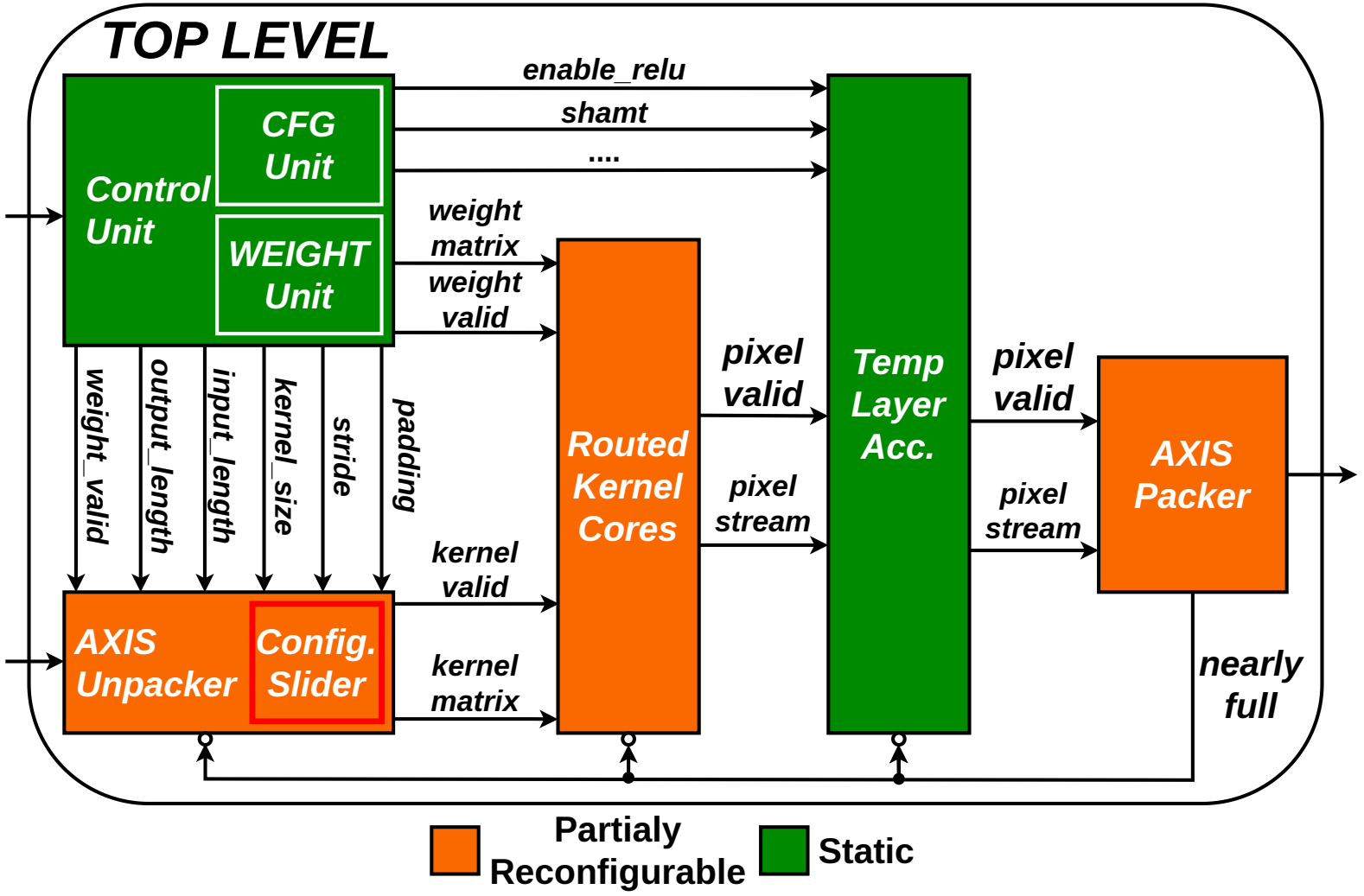




1. Introduction
2. Architecture Overview
3. Implementation
4. Evaluation
5. Conclusion
6. References

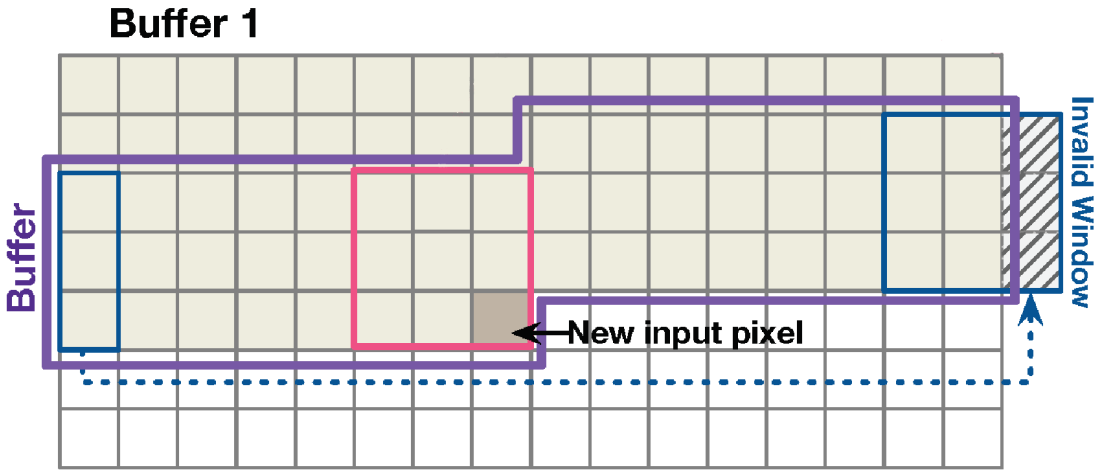
Accelerator Design

Structure of the most important submodules

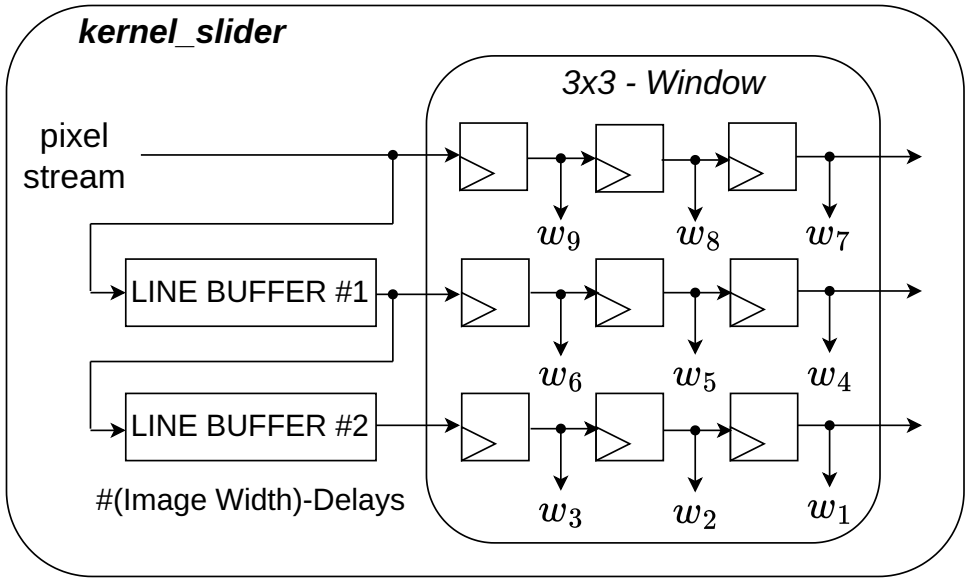


Configurable Slider

Undesired Static Approach



R. Shi, J. S. Wong, and H. K.-H. So. "High-Throughput Line Buffer Microarchitecture for Arbitrary Sized Streaming Image Processing". In: *Journal of Imaging* 5.3 (2019). DOI: 10.3390/jimaging5030034



L. Aranda, P. Reviriego, and J. A. Maestro. "Protecting Image Processing Pipelines against Configuration Memory Errors in SRAM-Based FPGAs". In: *Electronics* 7 (Nov. 2018), p. 322

Goal:

- Stream pixels one at a time into unit

⇒ Get all valid Kernel-Windows every other cycle

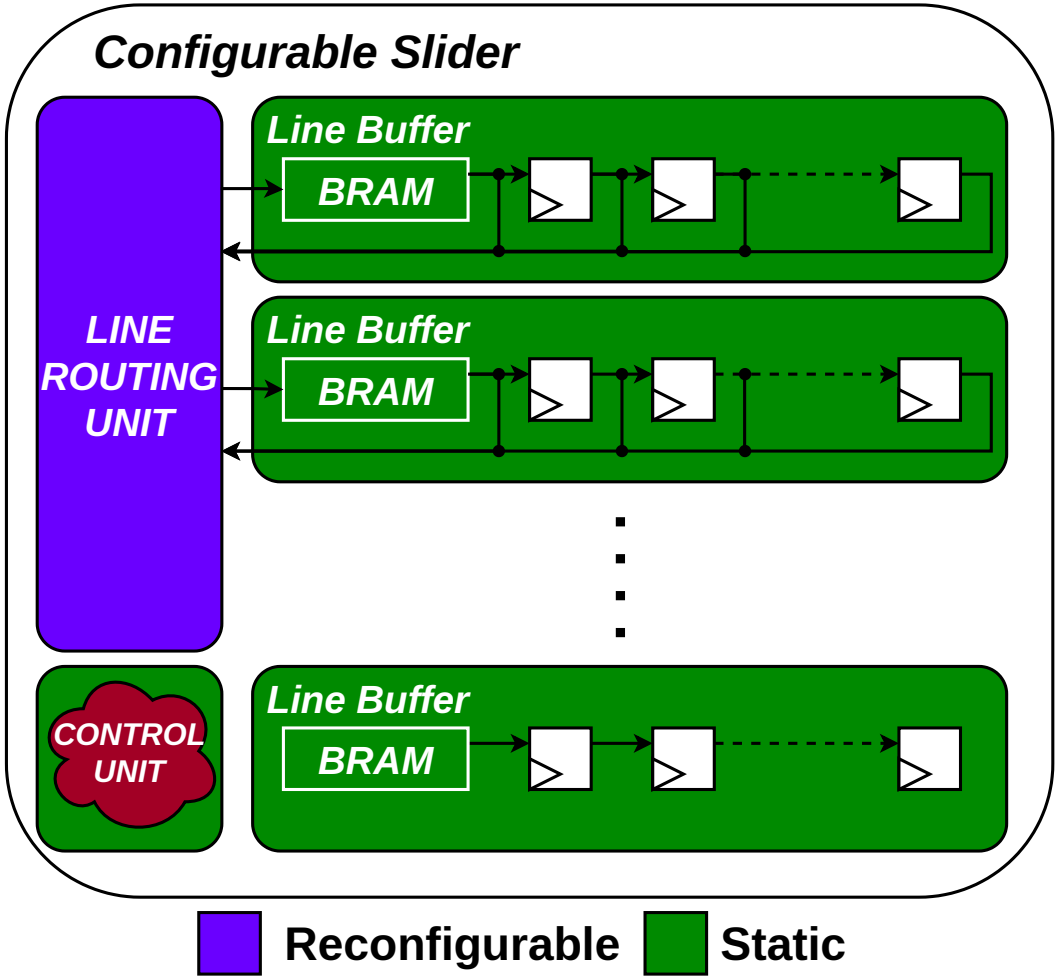
Problems:

- Only works with static **Kernel Size** and **IN-Channel Size**

⇒ Separate routing behavior and introduce extra logic

Configurable Slider

Partially Reconfigurable Approach



- Encapsulated routing inside separate **Line Routing Unit**

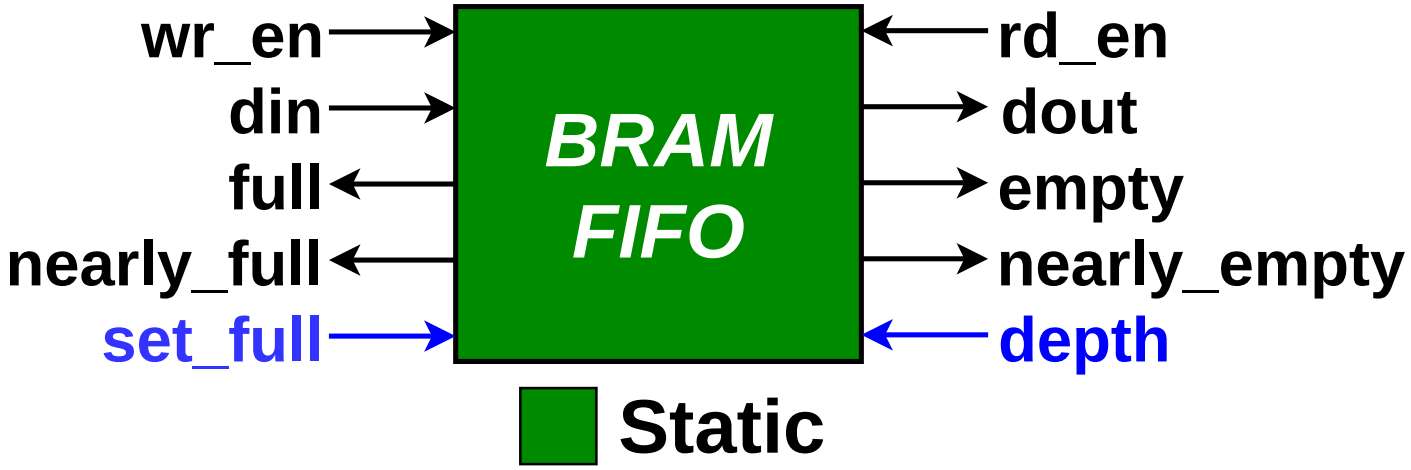
⇒ Each **Line Buffer** row as separate unit connected to routing

- Static, but dynamic, **Control Unit** to mark (in-)valid frames

⇒ *BRAM* tiles still fixed in size...

Configurable Slider

Dynamic BRAM Solution



- Depth of BRAM FIFO can be set during reset with newly **depth** port

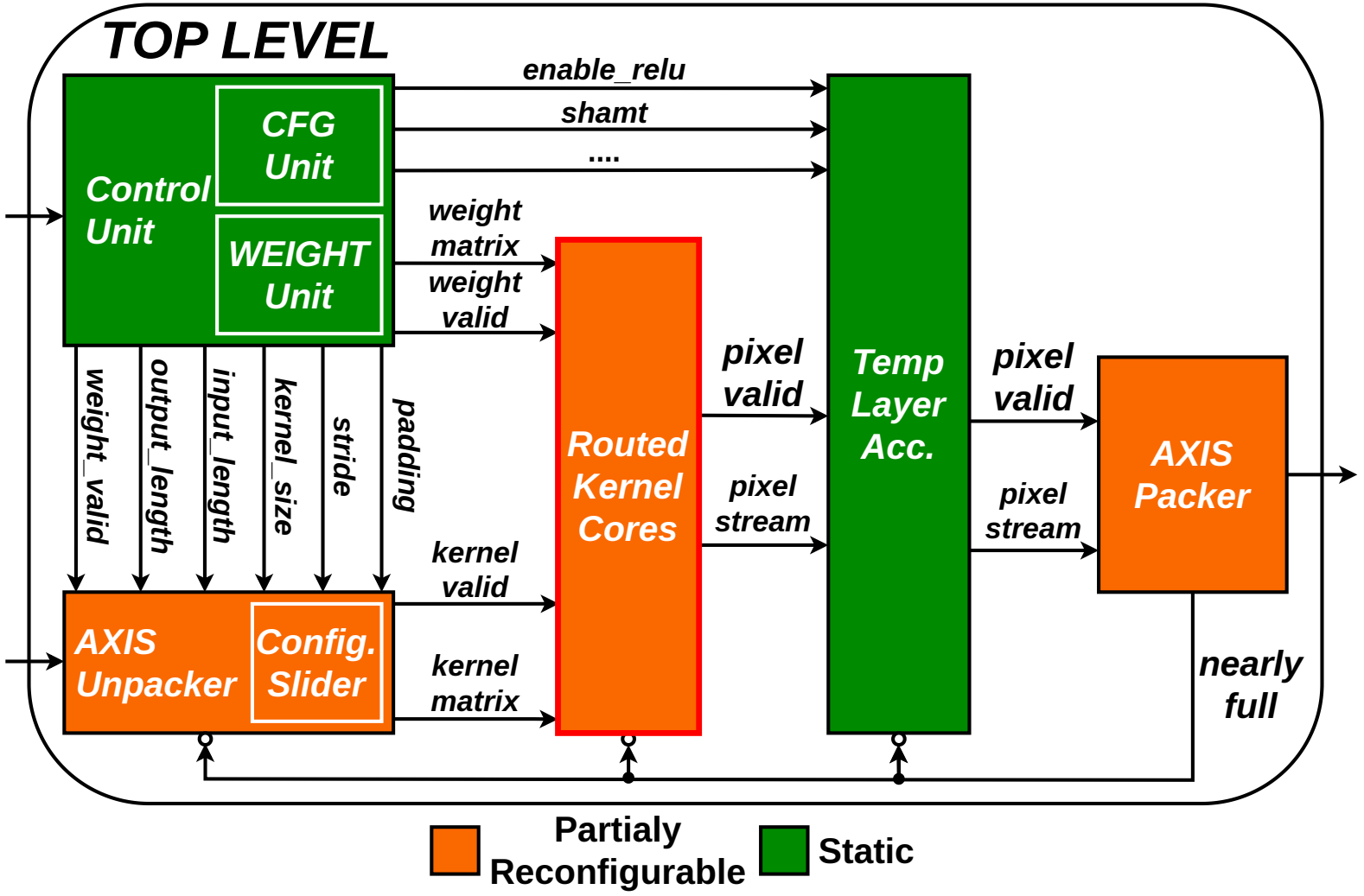
⇒ Depth is limited by a generic value set during instantiation ($depth \in [1, G_MAX_DEPTH]$)

- BRAM FIFO can also be set to full during reset with newly **set_full** port

⇒ Skipping initialization overhead when including *padding*

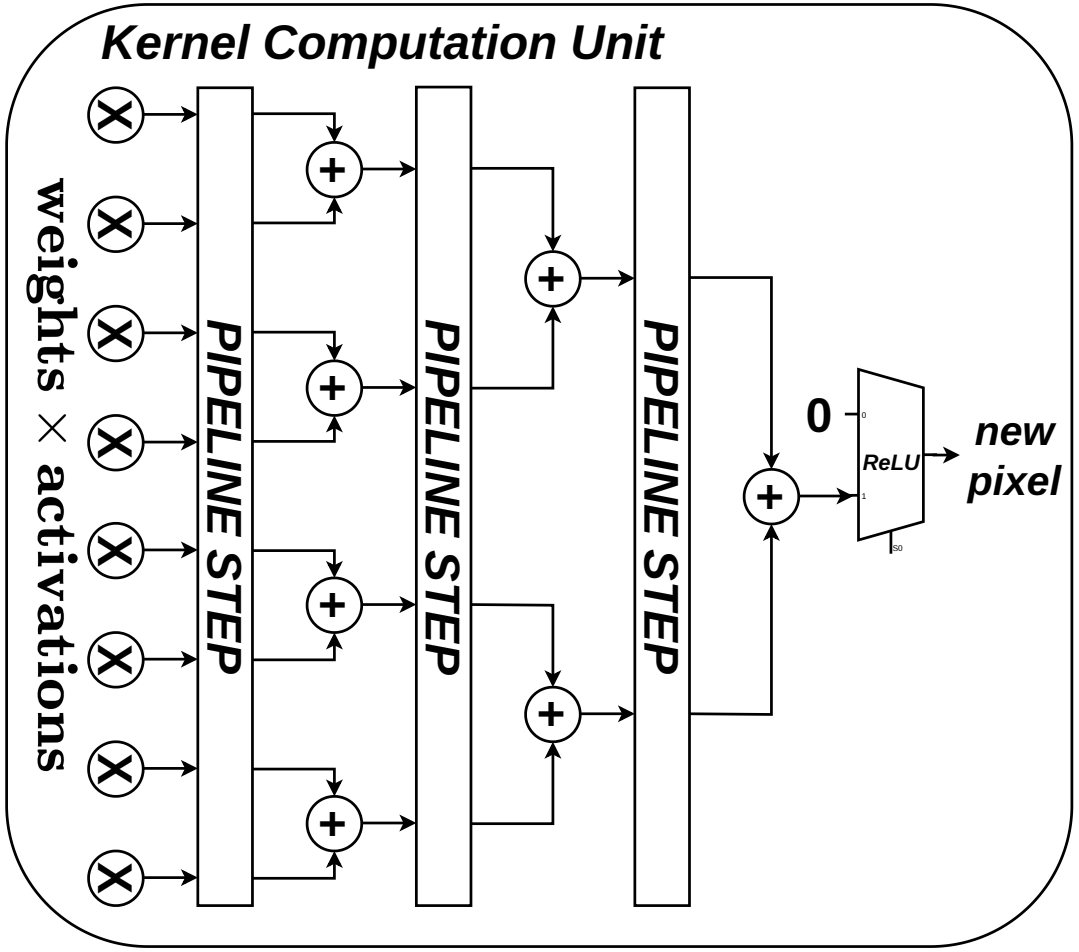
Accelerator Design

Structure of the most important submodules



Routed Kernel Cores

Static Tree Structures



First Idea:

- Pipelined tree structure to compute CNN layers

⇒ Simple design with high achievable frequencies

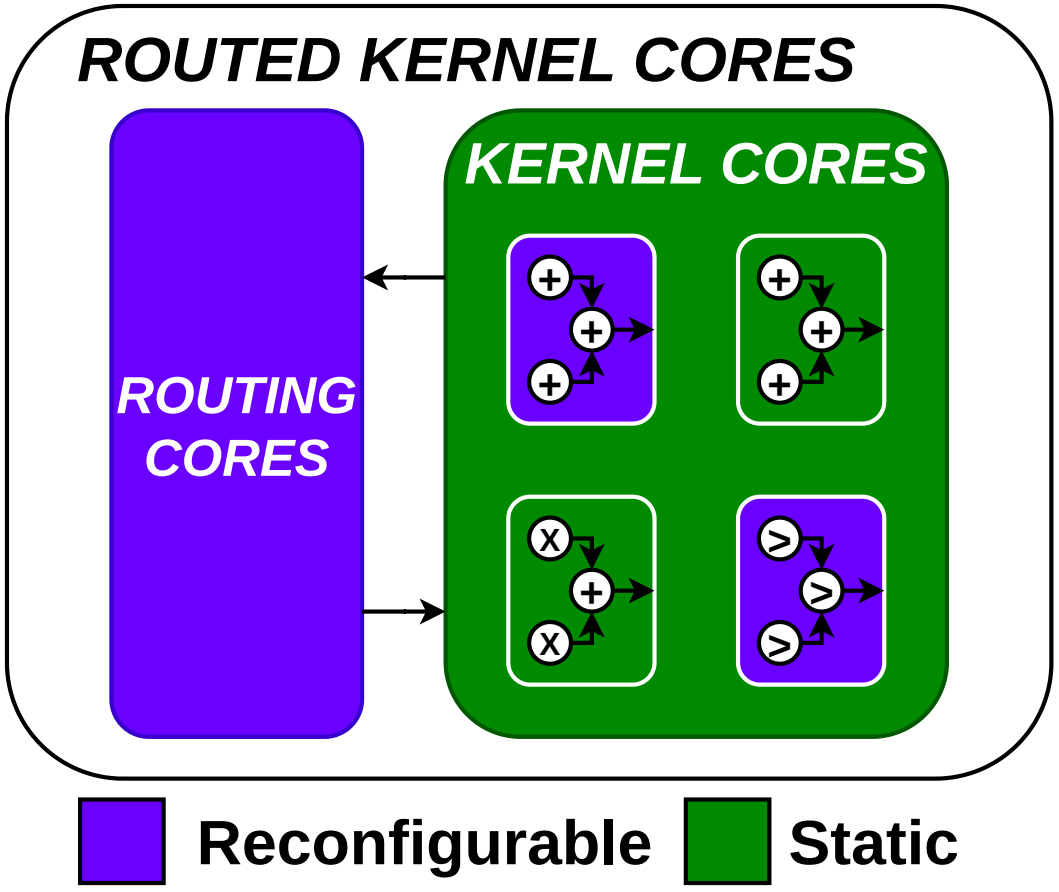
Problems:

- Routing is again completely static
- Trees with either *MAC* or comparison operations

⇒ Only possible to reconfigure entire region

Routed Kernel Cores

Partially Reconfigurable Approach



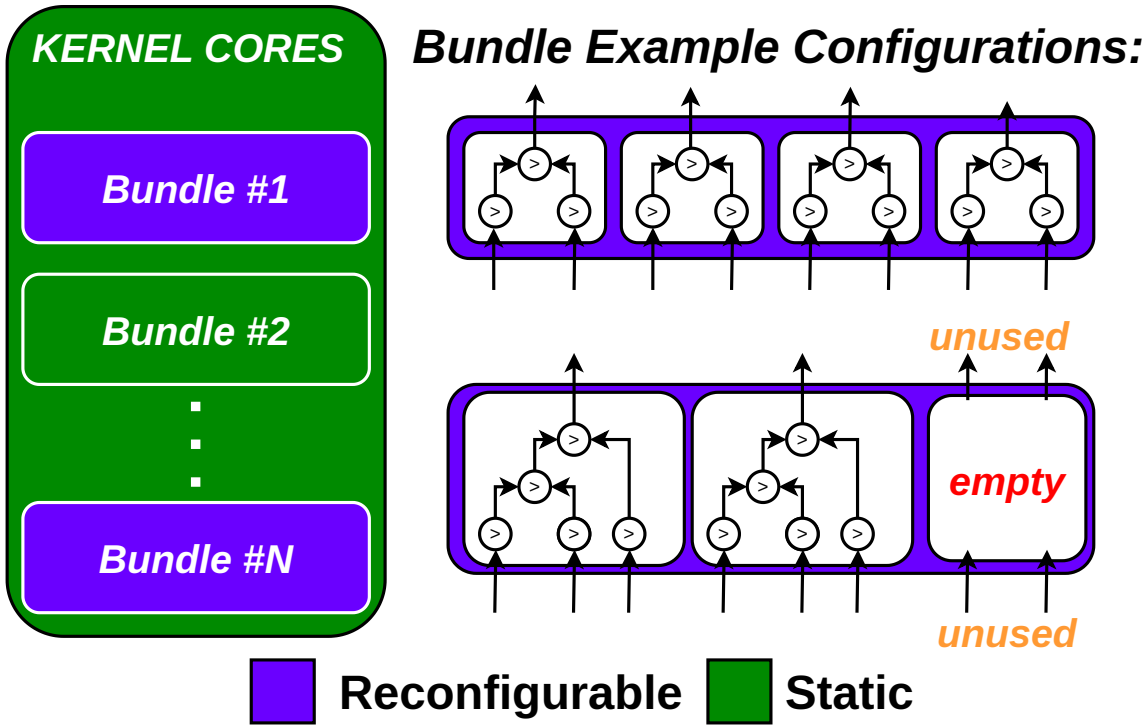
- Encapsulated routing inside separate **Routing Cores Unit**
- Tree structures encapsulated as bundles

⇒ Controllable, which bundles are static and which reconfigurable

⇒ Routing (and potentially) a few bundles need to be reconfigured

Routed Kernel Cores

Kernel Cores Bundle Abstraction



- Bundles limited by upper bounds for **IN** and **OUT** ports

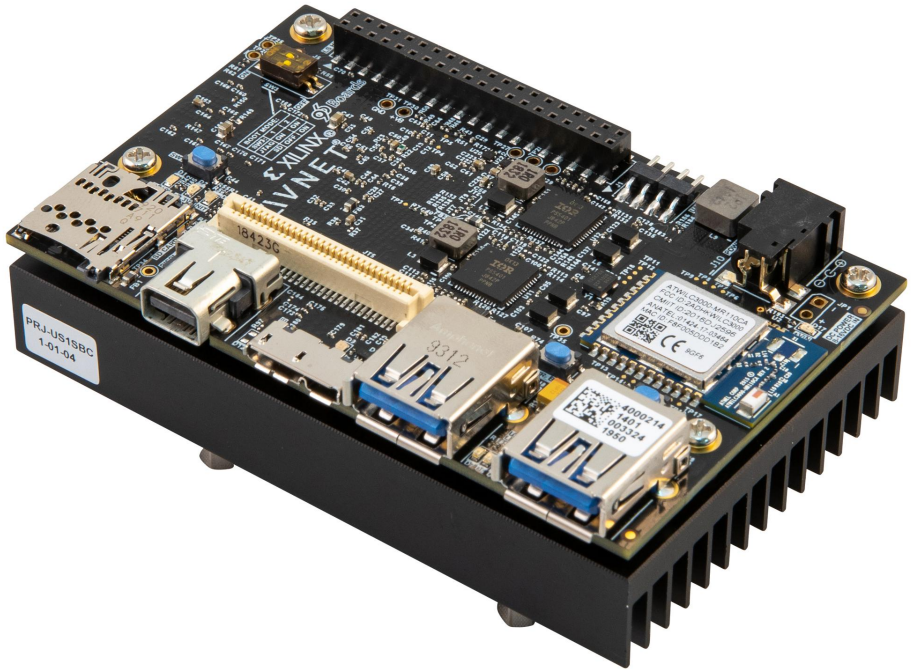
⇒ Granularity can be finely controlled by designer

⇒ Generation of all hardware modules via **Python-based code-generation framework**

1. Introduction
2. Architecture Overview
3. Implementation
4. Evaluation
5. Conclusion
6. References

Evaluation

Benchmarking Environments



FPGA Evaluation Platform:

- **Board:** Ultra96-v2
- **Arm-based Zynq Ultrascale+ MPSoC:**
  - **Processing System (PS):** Quad-core ARM Cortex-A53
  - **Programmable Logic (PL):**

CLB LUTs:	70.560
CLB FFs:	141.120
Block RAM Blocks:	216
LPDDR4 Frequency:	533 *MHz*

- **Application-Type:** Bare metal (no OS)

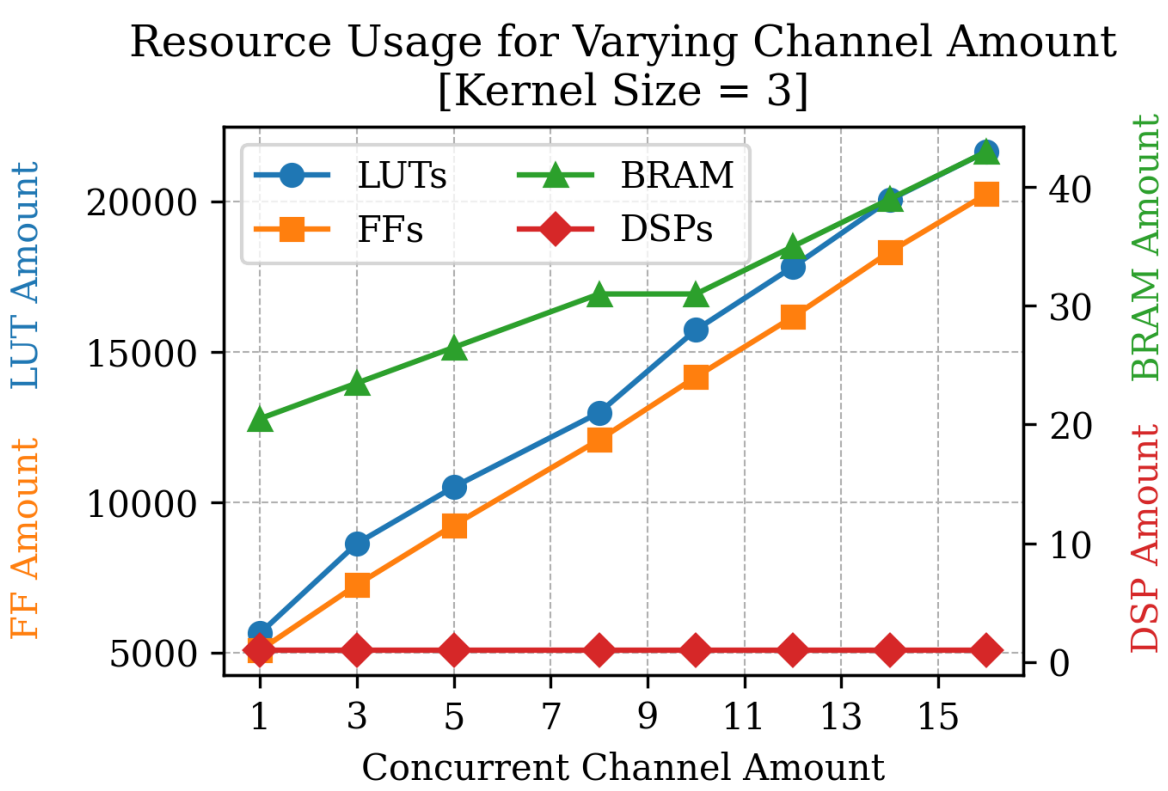
C/C++ Reference Platform (x86):

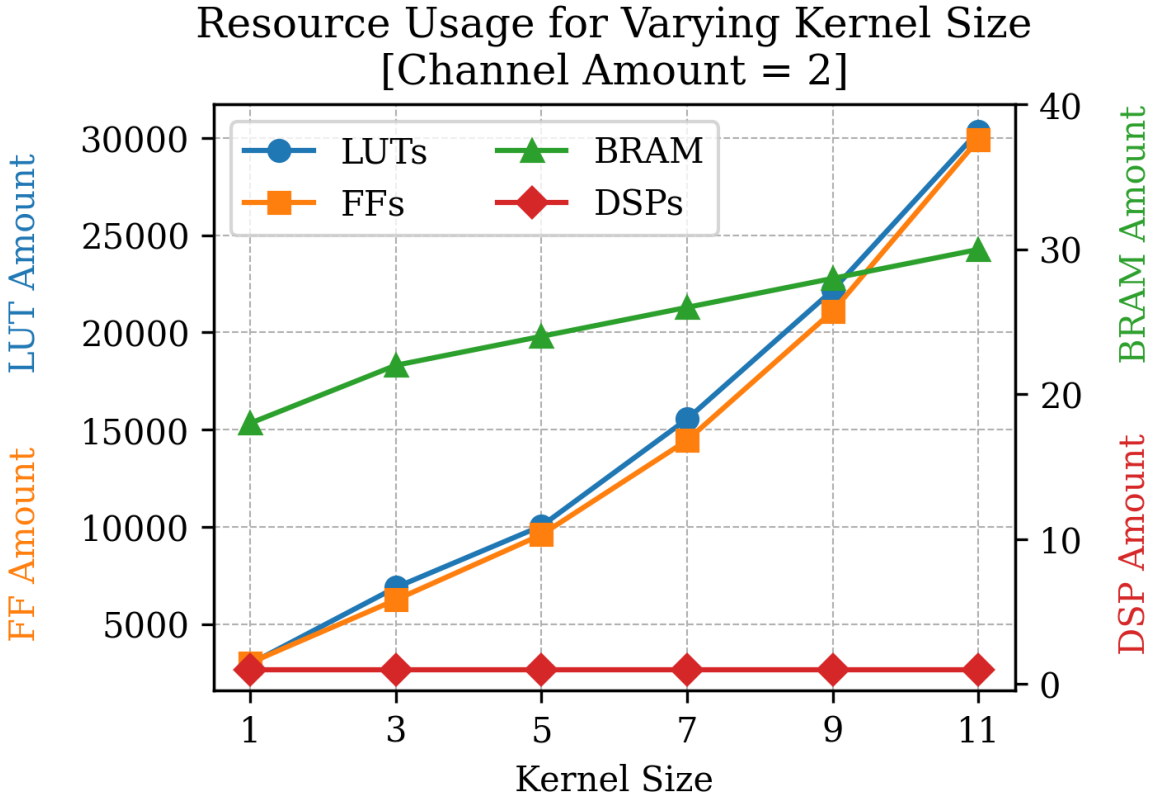
Processor:	*Intel Core i7-9750HF*
Clock Frequency:	2.6 – 4.5 *GHz*
DDR4 Frequency:	2400 *MHz*

Hardware Utilization

Scaling of Channel Amount and Kernel Size

Vivado *Out-Of-Context* Synthesis output:

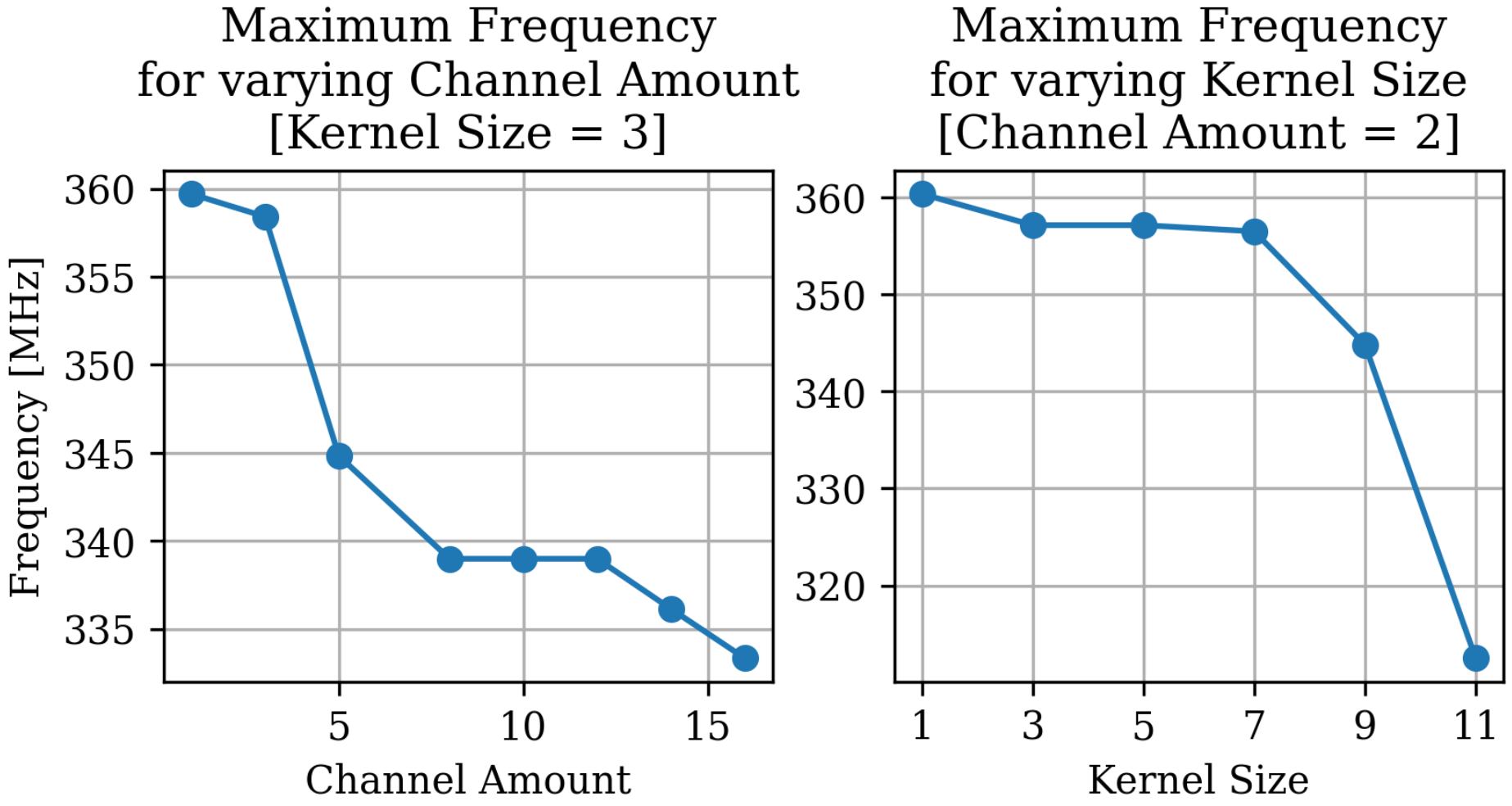




Achievable Frequencies

Scaling of Channel Amount and Kernel Size

Vivado *Out-Of-Context* Synthesis output:



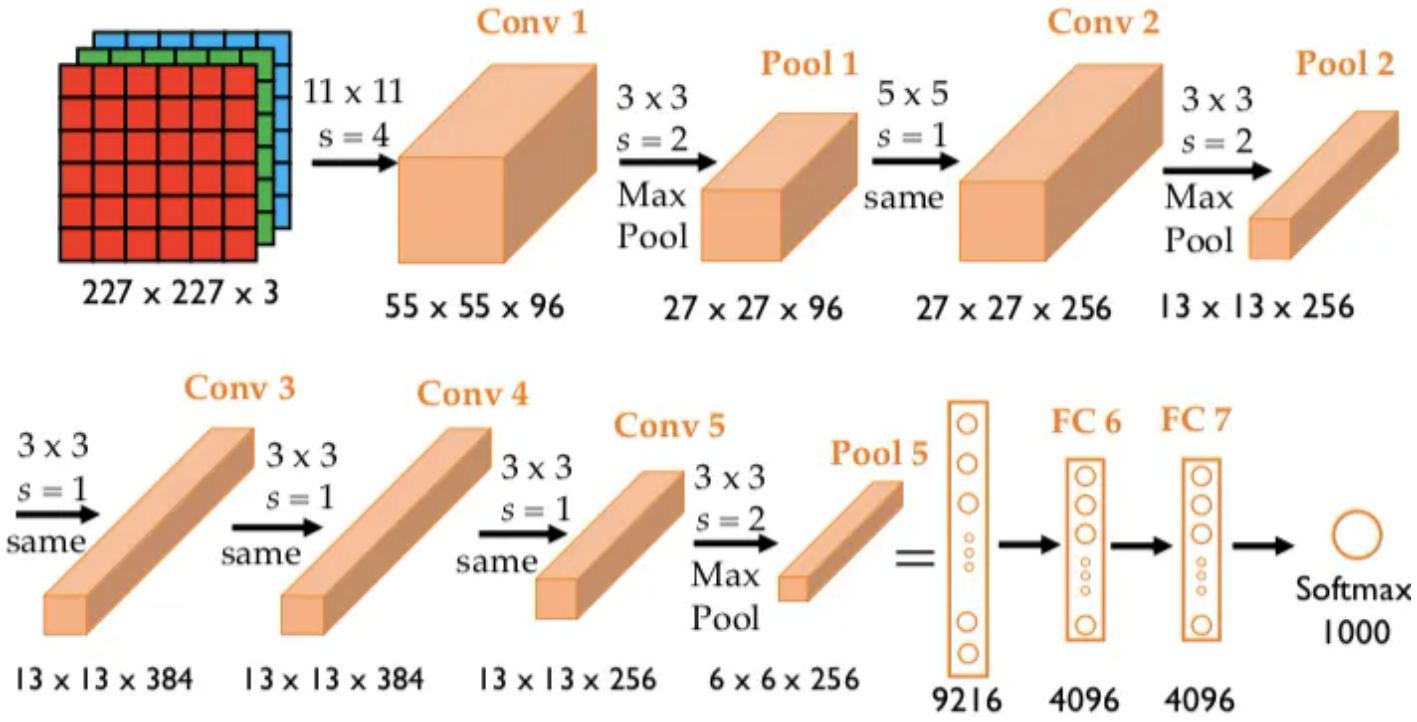
Hierarchical Resource Consumption

Convolution [*kernel_size* = 3; *channel_amount* = 8]

Instance (Accelerator)	LUTs	FFs	BRAM	DSPs
Top_Level_wrapper	19989 (28.33%)	23126 (16.39%)	38.5 (17.84%)	1 (0.28%)
top_level_cnn_wrapper	12205 (17.30%)	11198 (7.94%)	31 (14.15%)	1 (0.28%)
(top_level_cnn_wrapper)	792 (1.12%)	0 (0.00%)	0 (0.00%)	1 (0.28%)
top_level_cnn	11419 (16.18%)	11198 (7.94%)	31 (14.15%)	1 (0.28%)
axis_2_configurable_slider_wrapper	2801 (3.97%)	3031 (2.15%)	20 (9.26%)	0 (0.00%)
control_unit_axis_pipeline_step	326 (0.46%)	292 (0.21%)	0 (0.00%)	0 (0.00%)
control_unit	876 (1.24%)	1505 (1.07%)	0 (0.00%)	1 (0.28%)
pxs_2_axis	937 (1.33%)	724 (0.51%)	8 (3.70%)	0 (0.00%)
routed_kernel_cores	6388 (9.05%)	5474 (3.88%)	0 (0.00%)	0 (0.00%)
temp_layer_accumulator	120 (0.17%)	124 (0.09%)	3 (1.39%)	0 (0.00%)

Performance Evaluation

Comparison for AlexNet CNN Layers

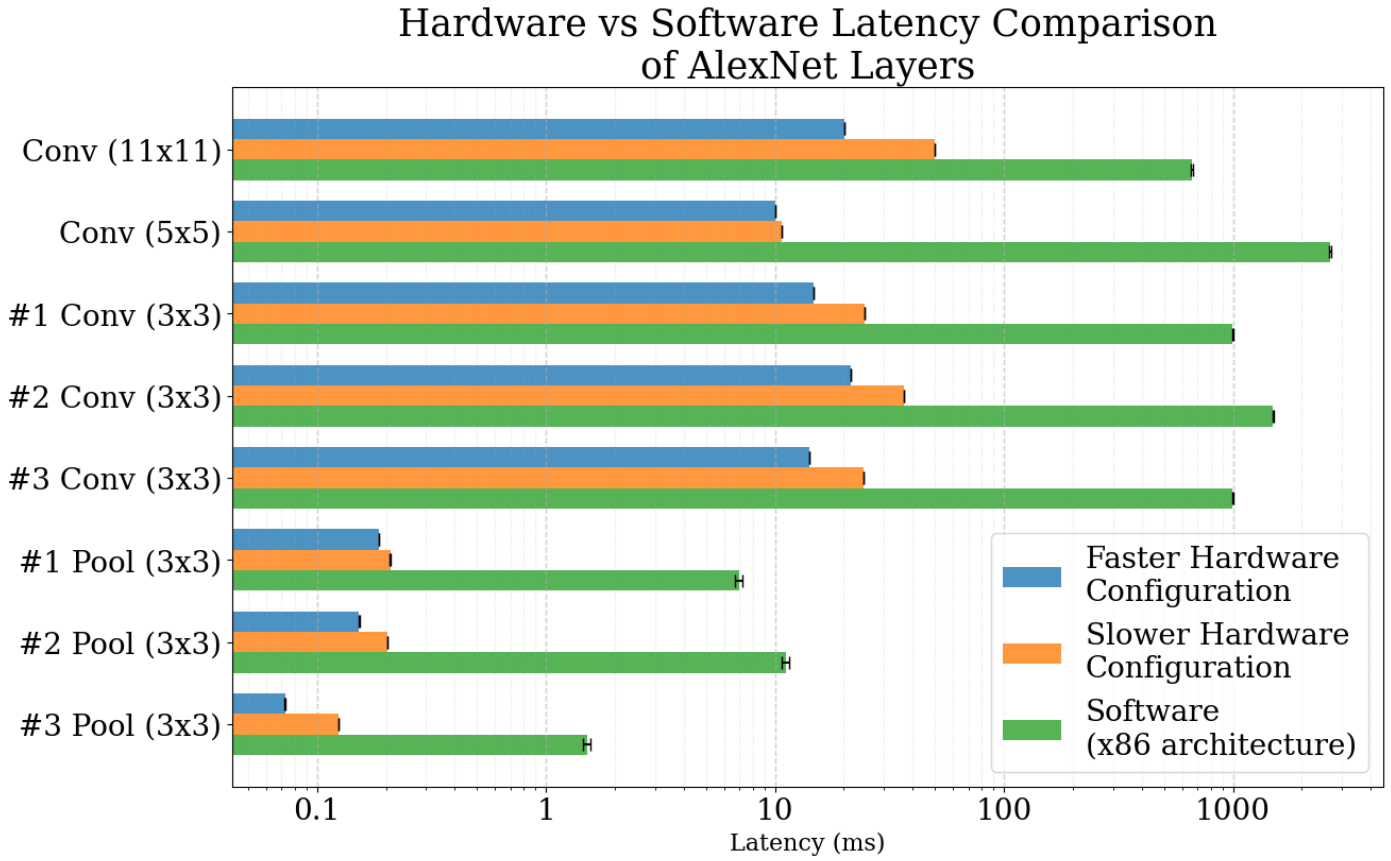


A. Krizhevsky, I. Sutskever, and G. E. Hinton. "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Ed. by F. Pereira, C. Burges, L. Bottou, and K. Weinberger. Vol. 25. Curran Associates, Inc., 2012

Layer Identifier	Input Channel	Output Channel	Channel Amount	Frequency (MHz)
Conv 11×11	3	96	Slow: 1 Fast: 3	300 250
Conv 5×5	96	256	Slow: 8 Fast: 16	300 250
Conv 3×3	256 384 384	384 384 256	Slow: 8 Fast: 16	300 250
Pool 3×3	96 256 256	96 256 256	Slow: 8 Fast: 16	300

Performance Evaluation

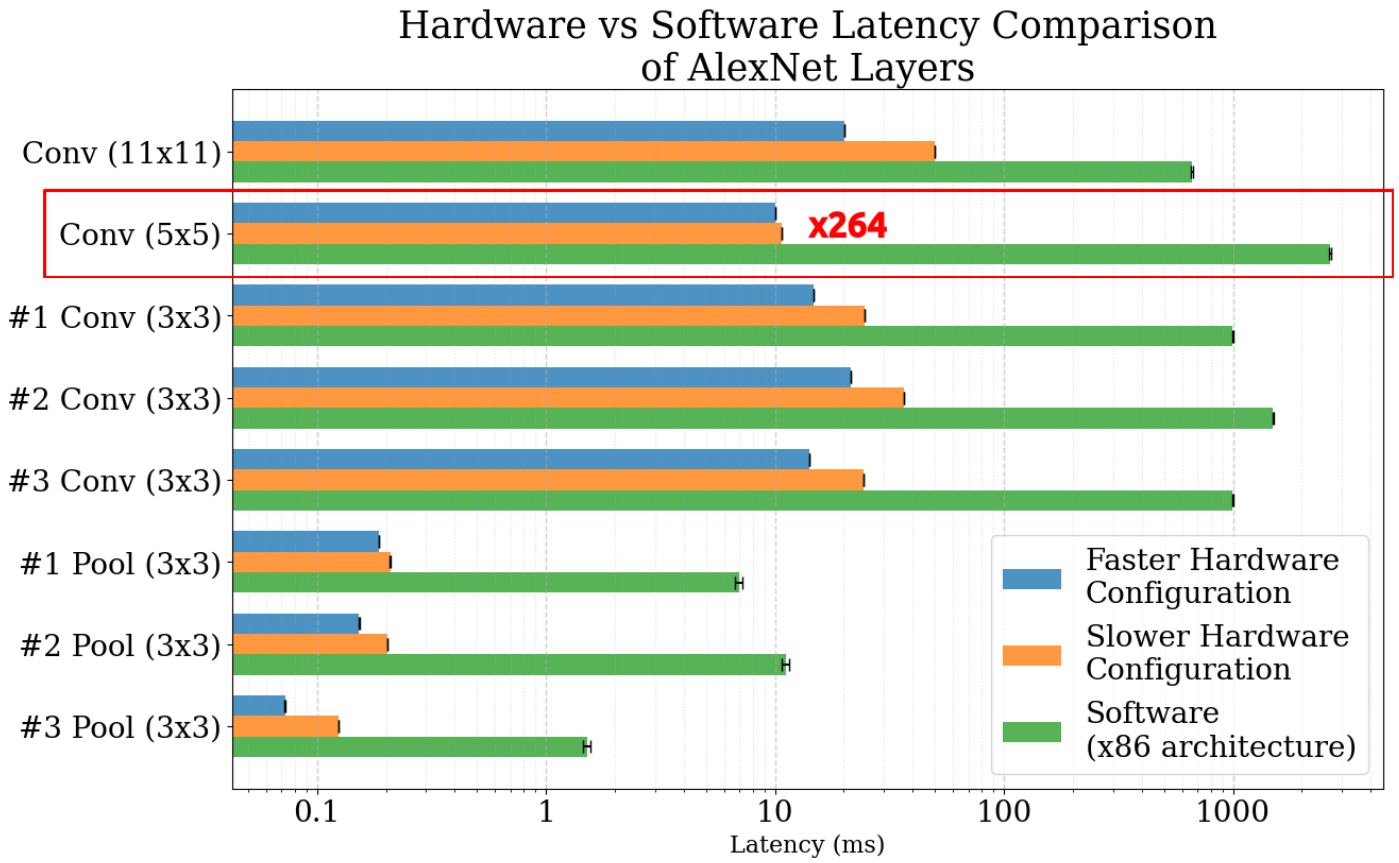
Comparison for AlexNet CNN Layers



Layer Identifier	Input Channel	Output Channel	Channel Amount	Frequency (MHz)
Conv 11×11	3	96	Slow: 1 Fast: 3	300 250
Conv 5×5	96	256	Slow: 8 Fast: 16	300 250
Conv 3×3	256 384 384	384 384 256	Slow: 8 Fast: 16	300 250
Pool 3×3	96 256 256	96 256 256	Slow: 8 Fast: 16	300

Performance Evaluation

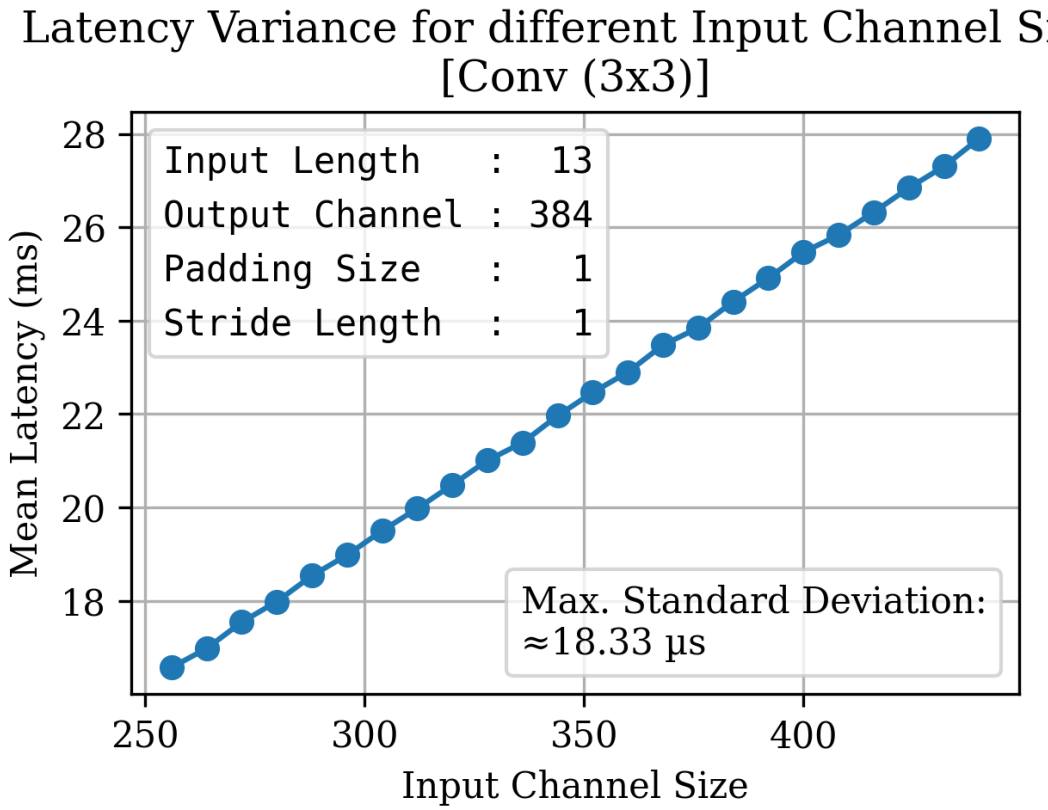
Comparison for AlexNet CNN Layers

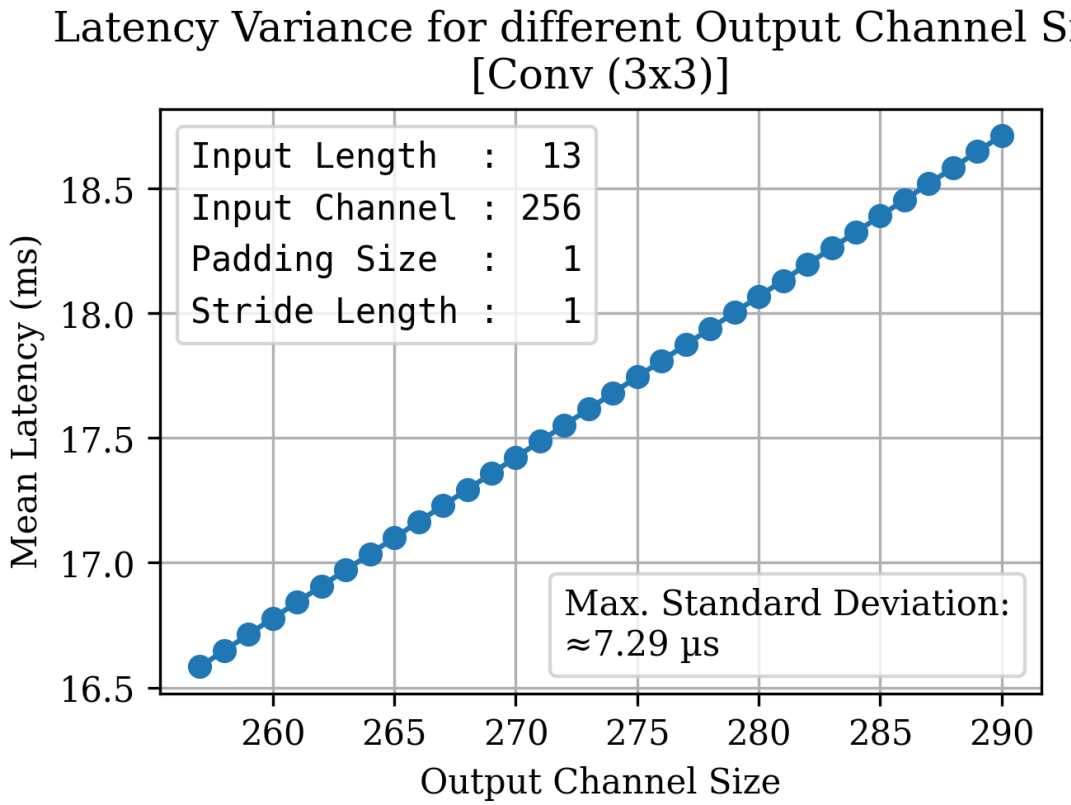


Layer Identifier	Input Channel	Output Channel	Channel Amount	Frequency (MHz)
Conv 11×11	3	96	Slow: 1 Fast: 3	300 250
Conv 5×5	96	256	Slow: 8 Fast: 16	300 250
Conv 3×3	256 384 384	384 384 256	Slow: 8 Fast: 16	300 250
Pool 3×3	96 256 256	96 256 256	Slow: 8 Fast: 16	300

Performance Evaluation

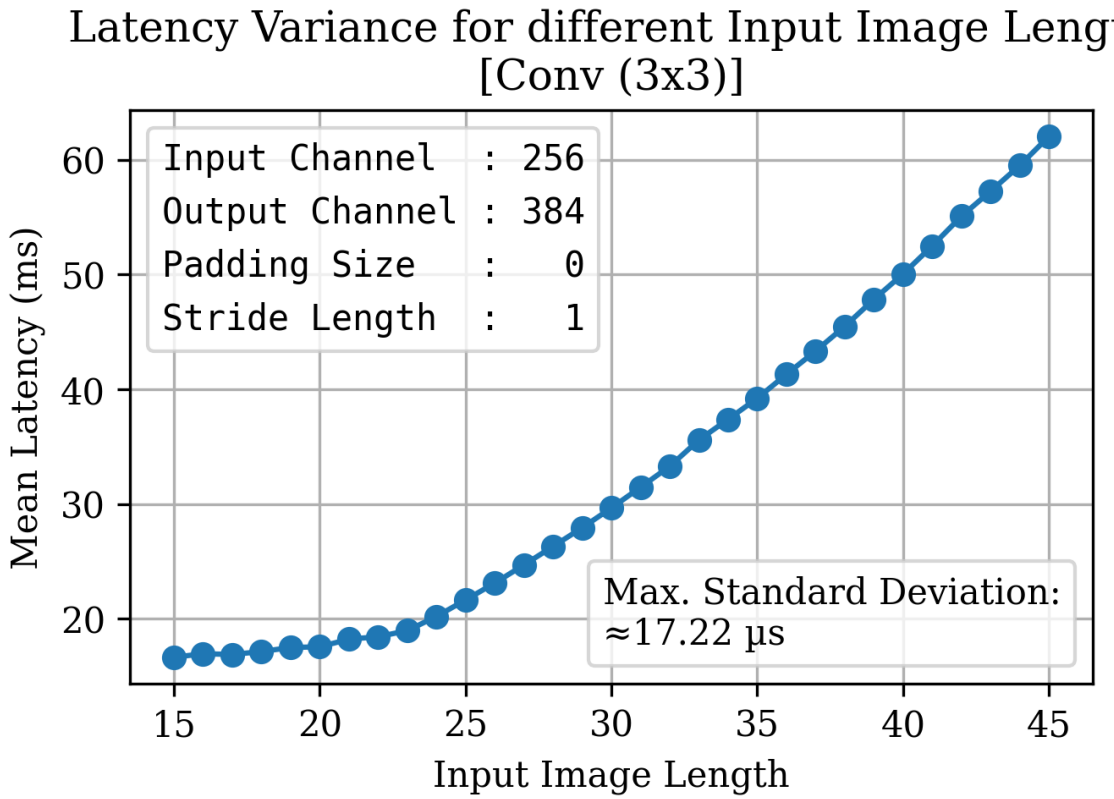
Scaling of Input and Output Channel Sizes

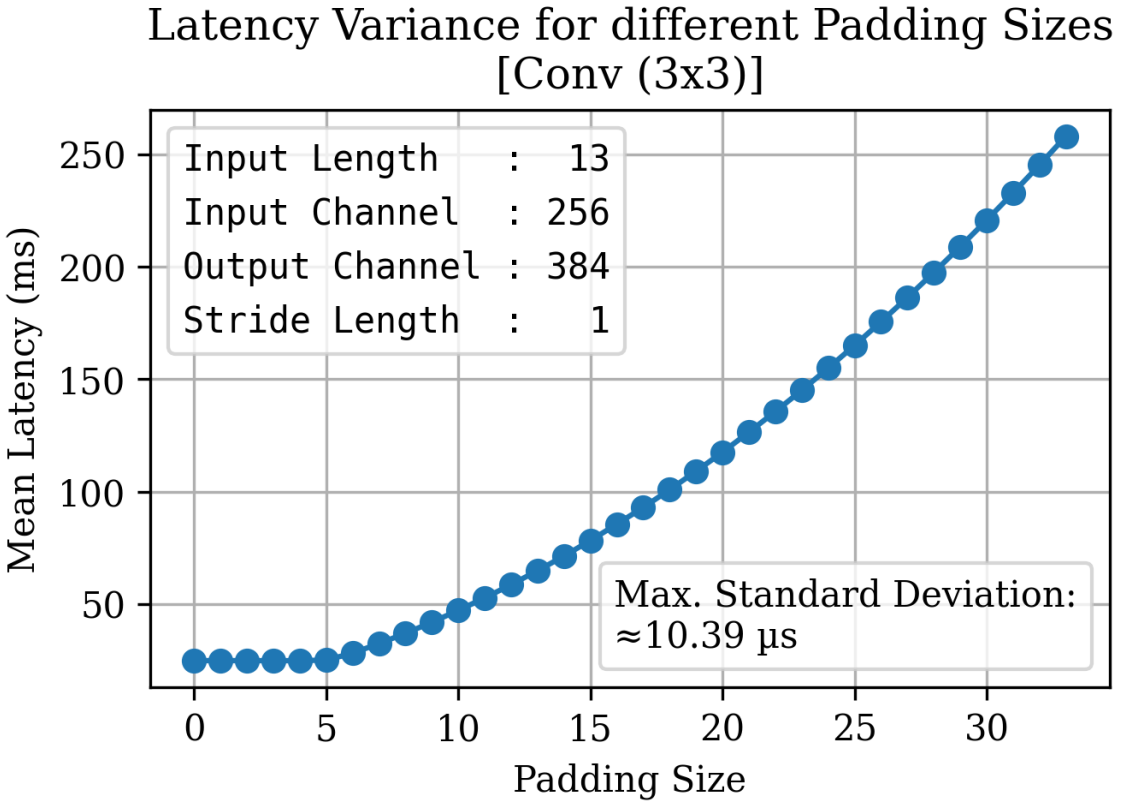




Performance Evaluation

Scaling of Input and Padding Sizes





1. Introduction
2. Architecture Overview
3. Implementation
4. Evaluation
5. Conclusion
6. References

Conclusion

Summary and Future Work

Achievements:

- Generic, highly customizable and *partially reconfigurable* CNN hardware accelerator
- Tooling to streamline and automate the hardware generation process
- Best-Case speedup factor of up to $\times$**264** against an **x86**-Architecture running a C/C++ reference implementation

Future Work:

- Measurement of actual reconfiguration times across various FPGA architectures
- Evaluation of performance for entire CNN architectures (include Fully-Connected-Layers)
- Explore other quantization Bit-Widths
- (. . .)

Thank you for your attention!

Any questions?

1. Introduction
2. Architecture Overview
3. Implementation
4. Evaluation
5. Conclusion
6. References

References

[ARM18] L. Aranda, P. Reviriego, and J. A. Maestro. “Protecting Image Processing Pipelines against Configuration Memory Errors in SRAM-Based FPGAs”. In: *Electronics* 7 (Nov. 2018), p. 322.

[BXZ20] Y. Bi, B. Xue, and M. Zhang. “Evolving Deep Forest with Automatic Feature Extraction for Image Classification Using Genetic Programming”. In: Aug. 2020, pp. 3–18. DOI: 10.1007/978-3-030-58112-1_1.

[He+16] K. He, X. Zhang, S. Ren, and J. Sun. “Deep Residual Learning for Image Recognition”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90.

[KSH12] A. Krizhevsky, I. Sutskever, and G. E. Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems*. Ed. by F. Pereira, C. Burges, L. Bottou, and K. Weinberger. Vol. 25. Curran Associates, Inc., 2012.

[RW23] E. Rex and X. Wang. “Efficient Quantization and Hardware Implementation of AlexNet on Resource-limited FPGAs”. In: *2023 IEEE 14th Annual Ubiquitous Computing, Electronics and Mobile Communication Conference (UEMCON)*. 2023, pp. 0399–0406. DOI: 10.1109/UEMCON59035.2023.10316010.

[SWS19] R. Shi, J. S. Wong, and H. K.-H. So. “High-Throughput Line Buffer Microarchitecture for Arbitrary Sized Streaming Image Processing”. In: *Journal of Imaging* 5.3 (2019). DOI: 10.3390/jimaging5030034.

7. Related Work

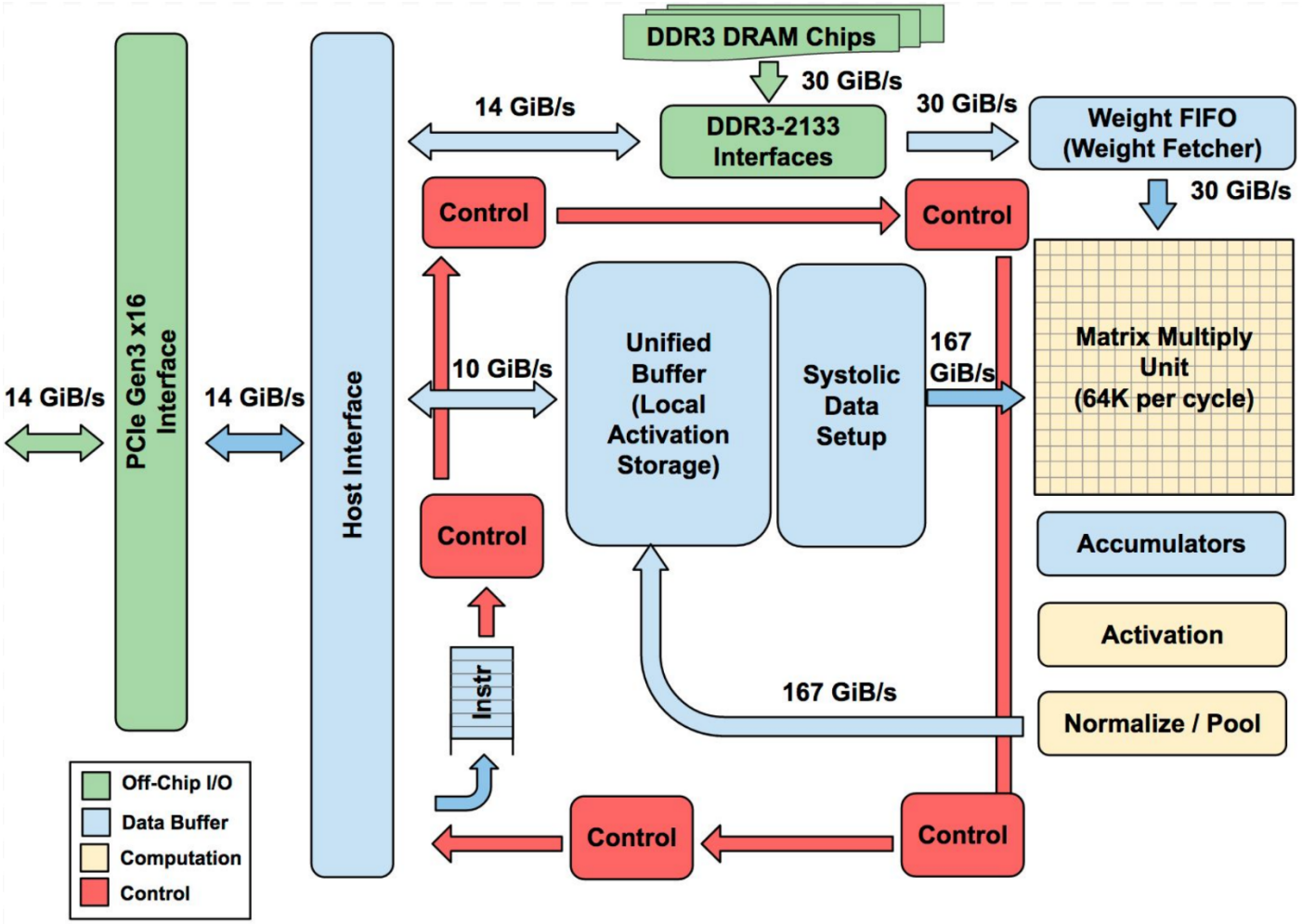
8. Tooling

9. Further Implementation

10. Further Evaluation

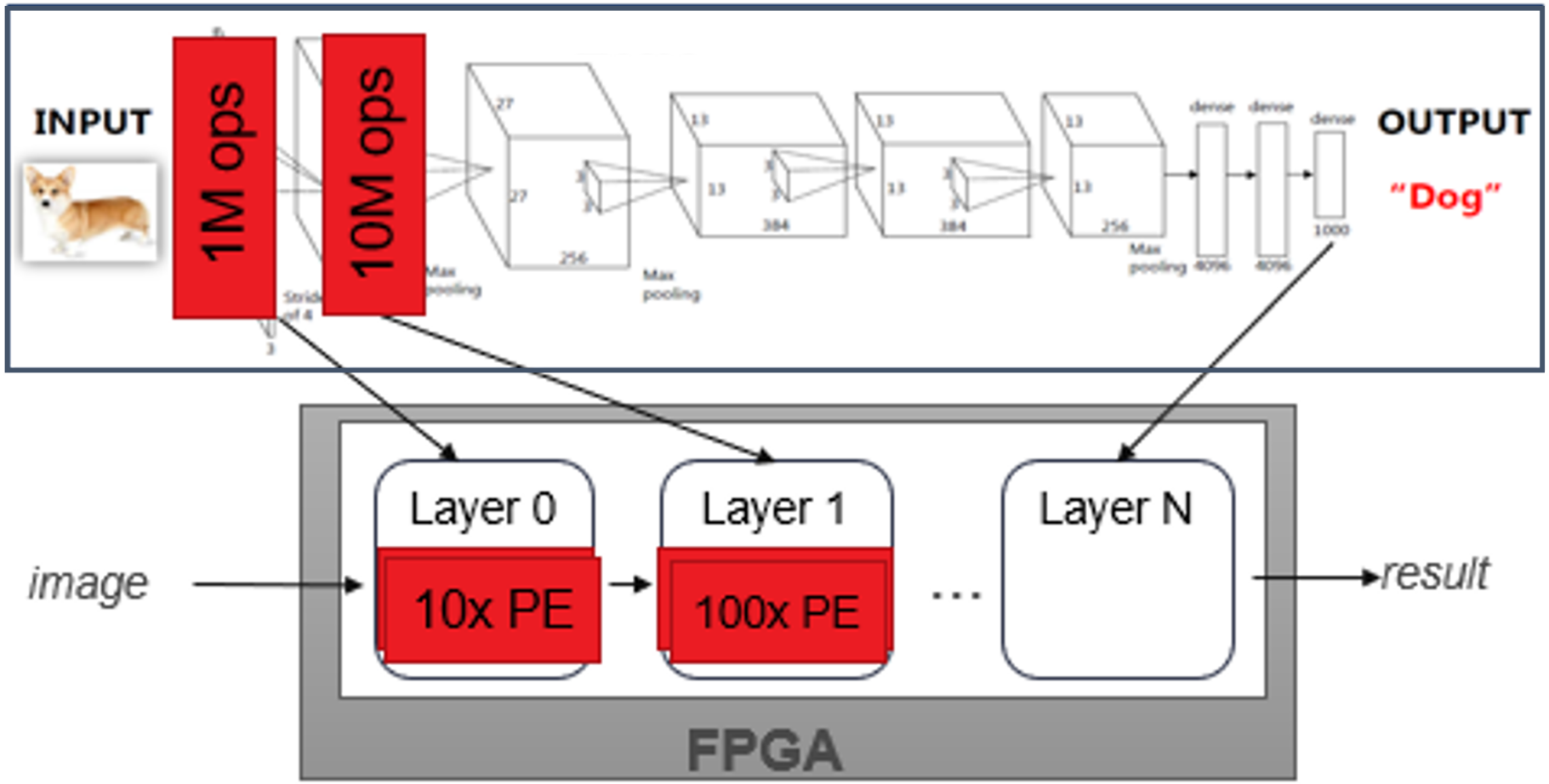
Related Works

Google's Tensor Processing Unit (TPU)



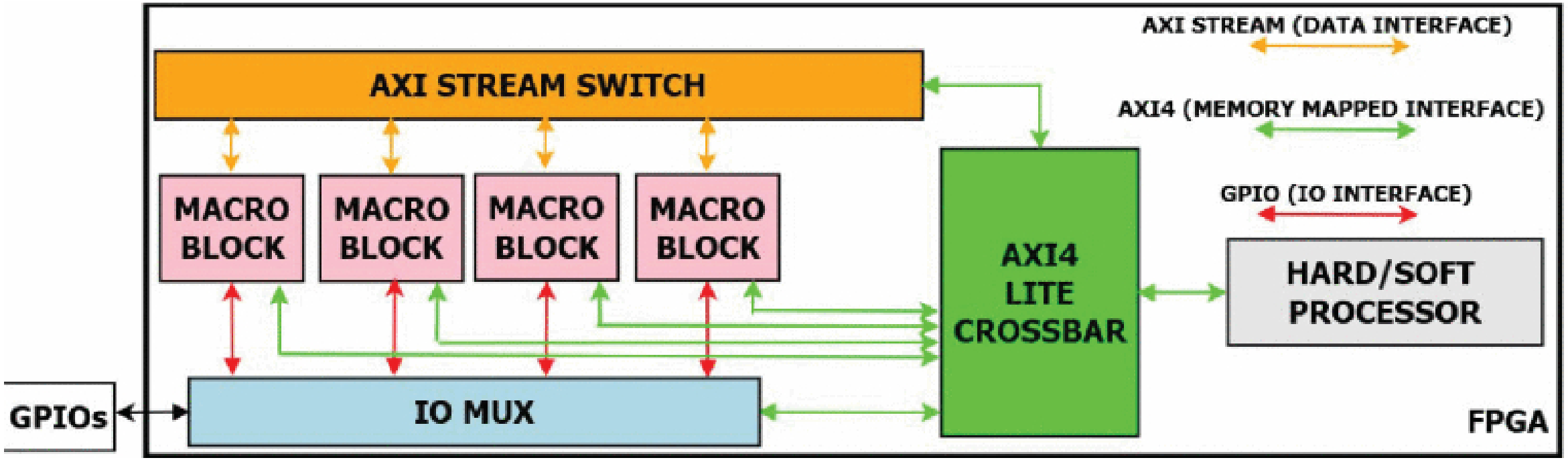
Related Works

AMD's FINN Compiler



Related Works

MacroBlock based Partially Reconfigurable Design



7. Related Work

8. Tooling

9. Further Implementation

10. Further Evaluation

Tooling

Training Evaluation and Comparison

Network	Implementation	Data-Type	Epochs	Accuracy
ResNet	Deep Residual Learning for Image Recognition [BXZ20; He+16]	*float32*	N/A	93.57%
AlexNet	ImageNet Classification with Deep CNNs [BXZ20; KSH12]	*float32*	N/A	83.00%
AlexNet	Efficient Quantization and HW Implementation of AlexNet on Resource-limited FPGAs [RW23]	*int8*	250	83.16%
AlexNet	*Part. Reconfigurable Architecture (Python and C/C++)*	*int8*	*100*	*76.67%*
AlexNet	Efficient Quantization and HW Implementation of AlexNet on Resource-limited FPGAs [RW23]	*int1*	250	72.66%



7. Related Work

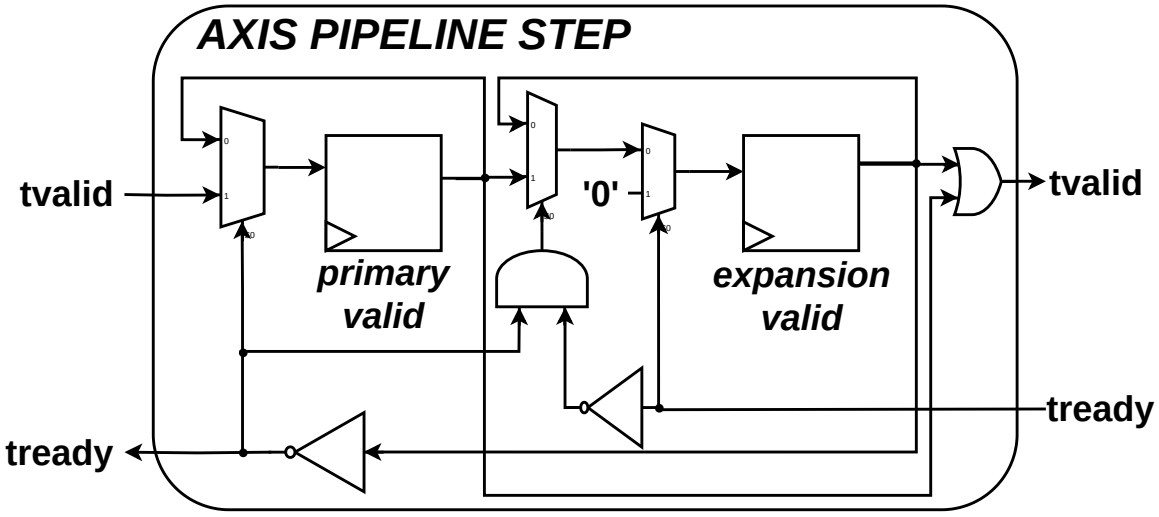
8. Tooling

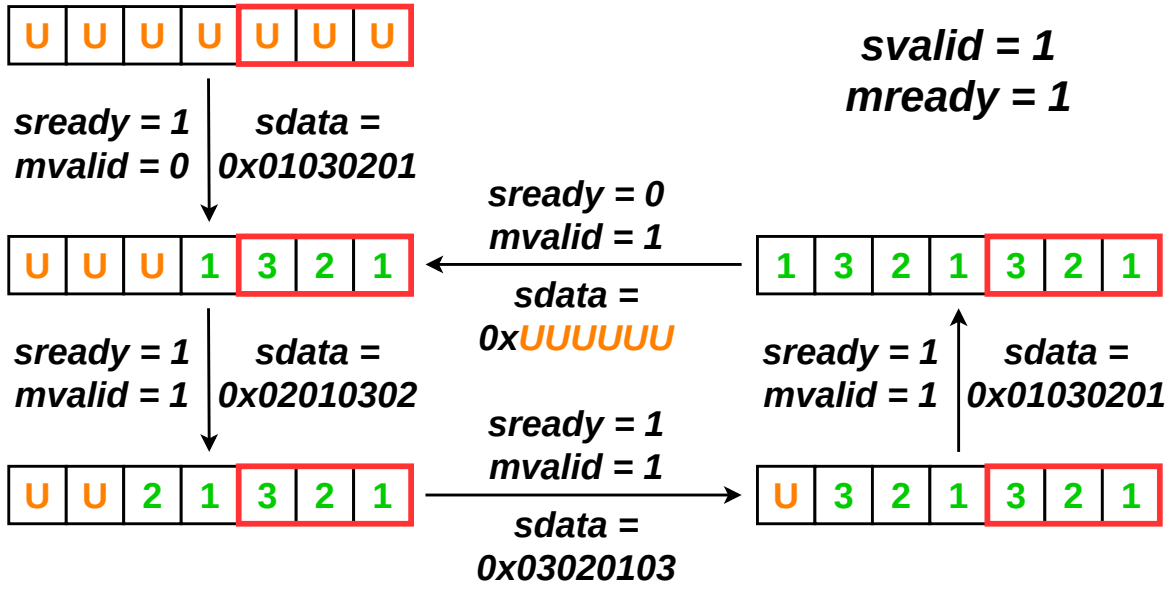
9. Further Implementation

10. Further Evaluation

AXI-Stream Components

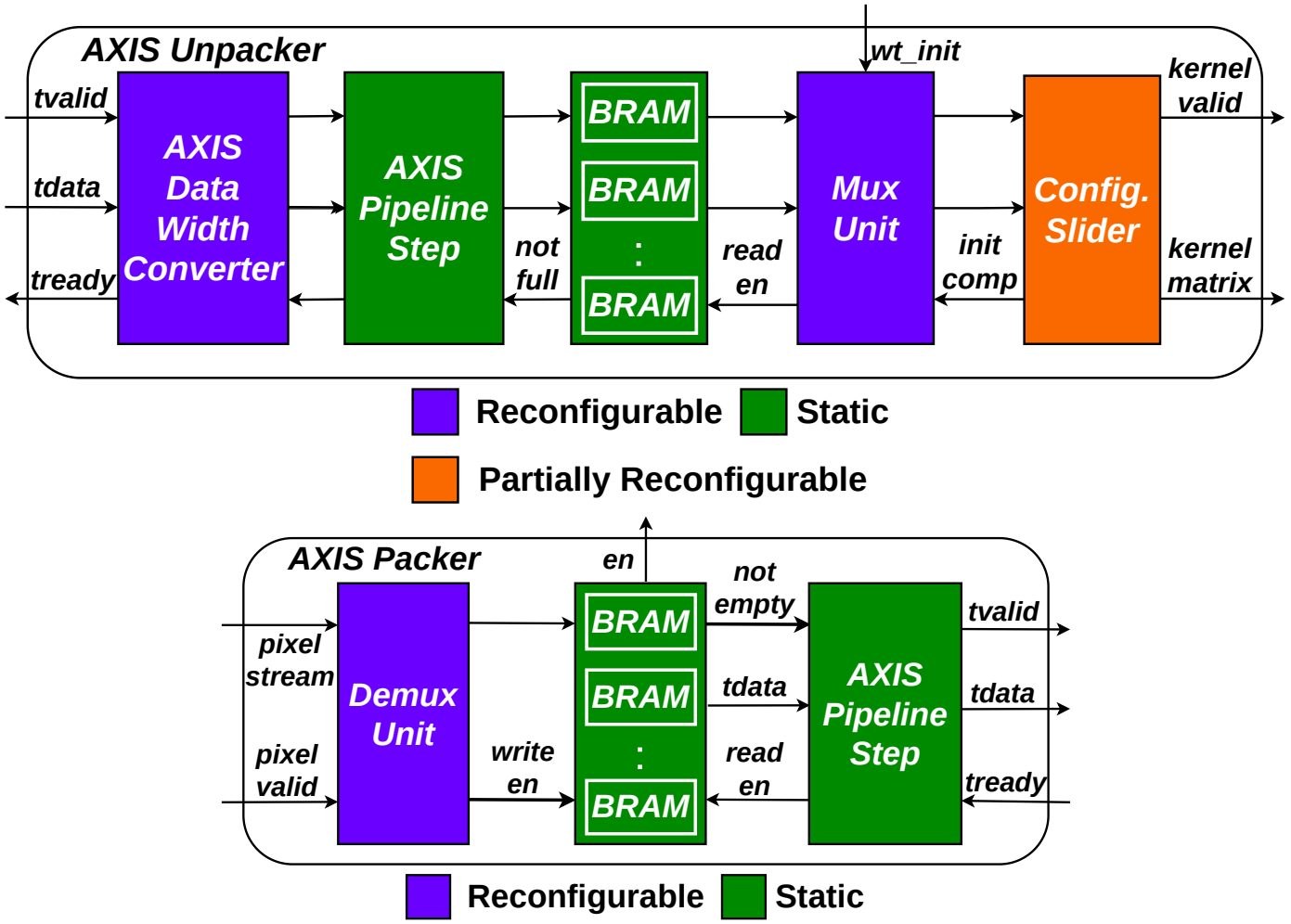
Pipelining and Downsizing





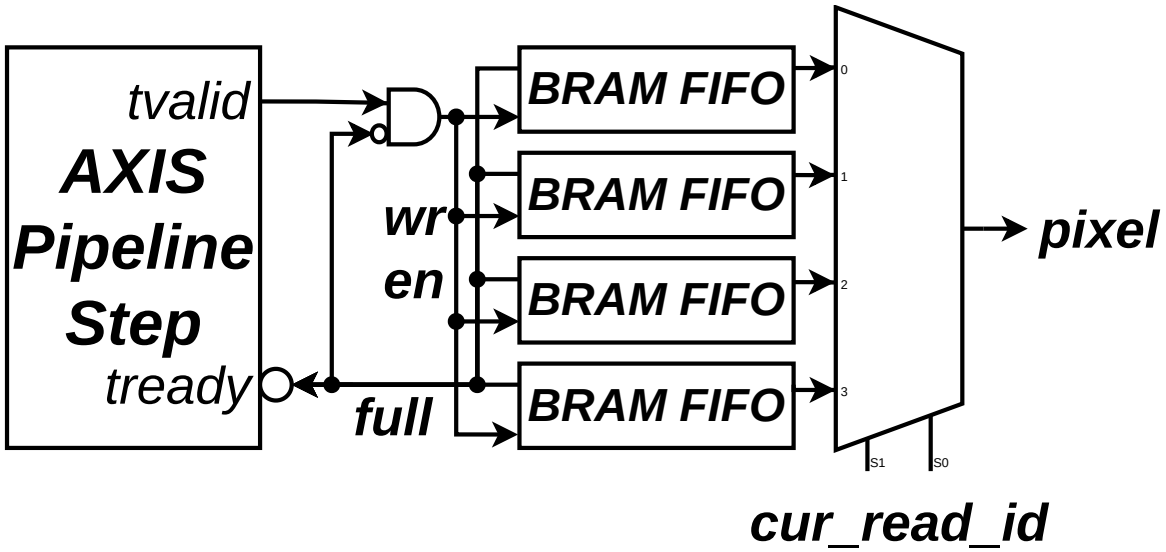
AXI-Stream Components

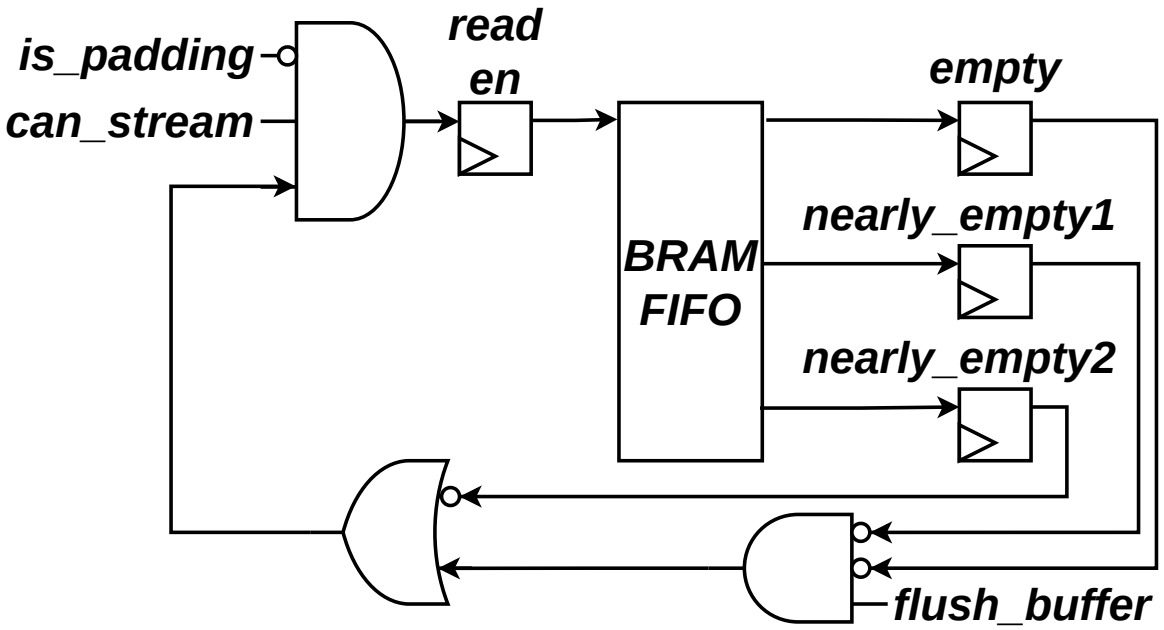
AXIS (Un-)Packer Units



AXI-Stream Components

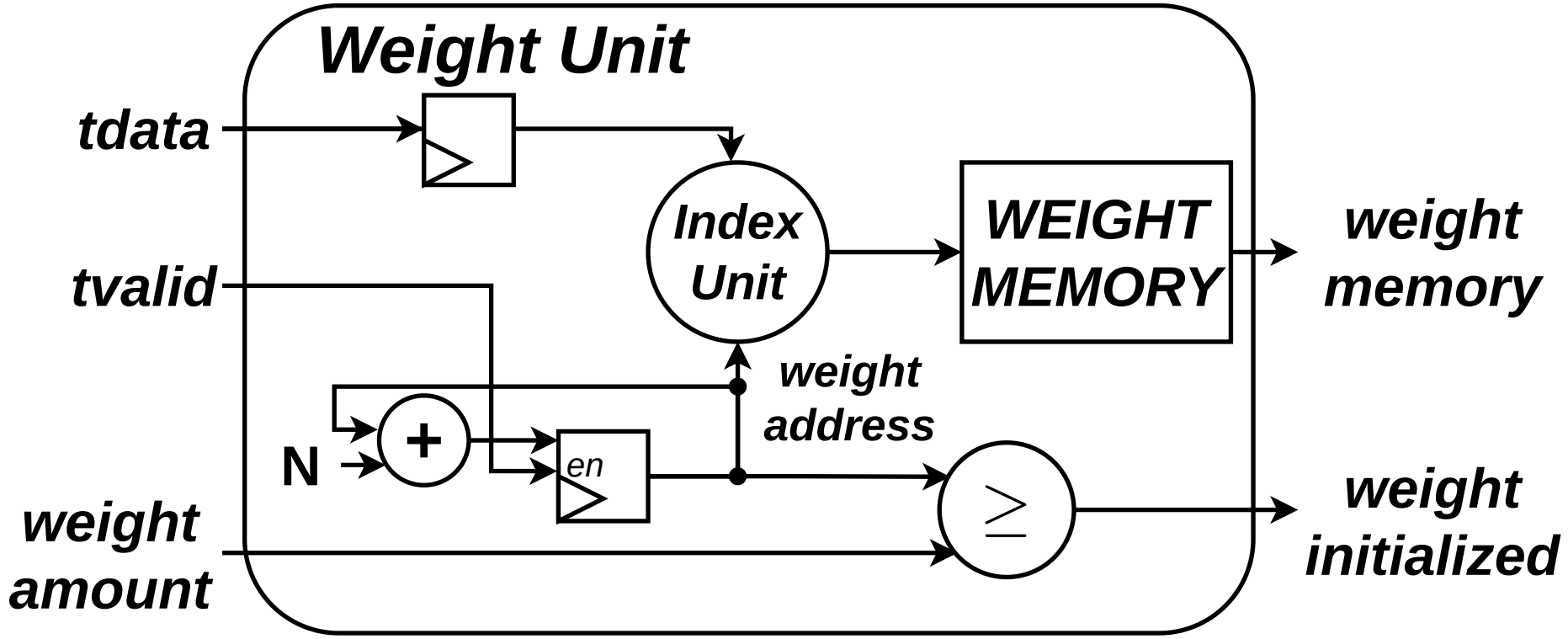
Hazard Control





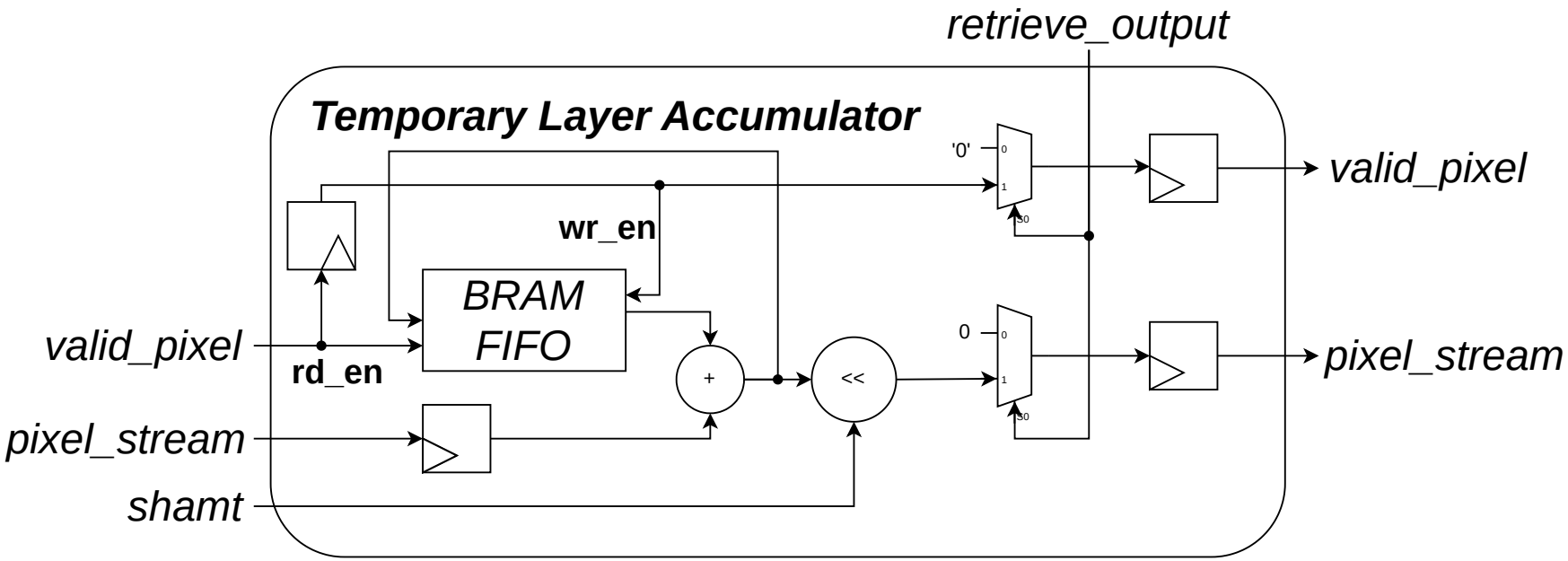
Weight Unit

Retrieval and Storing Process for Weight Data



Temporary Layer Accumulator

Iterative Computation of CNN Layers



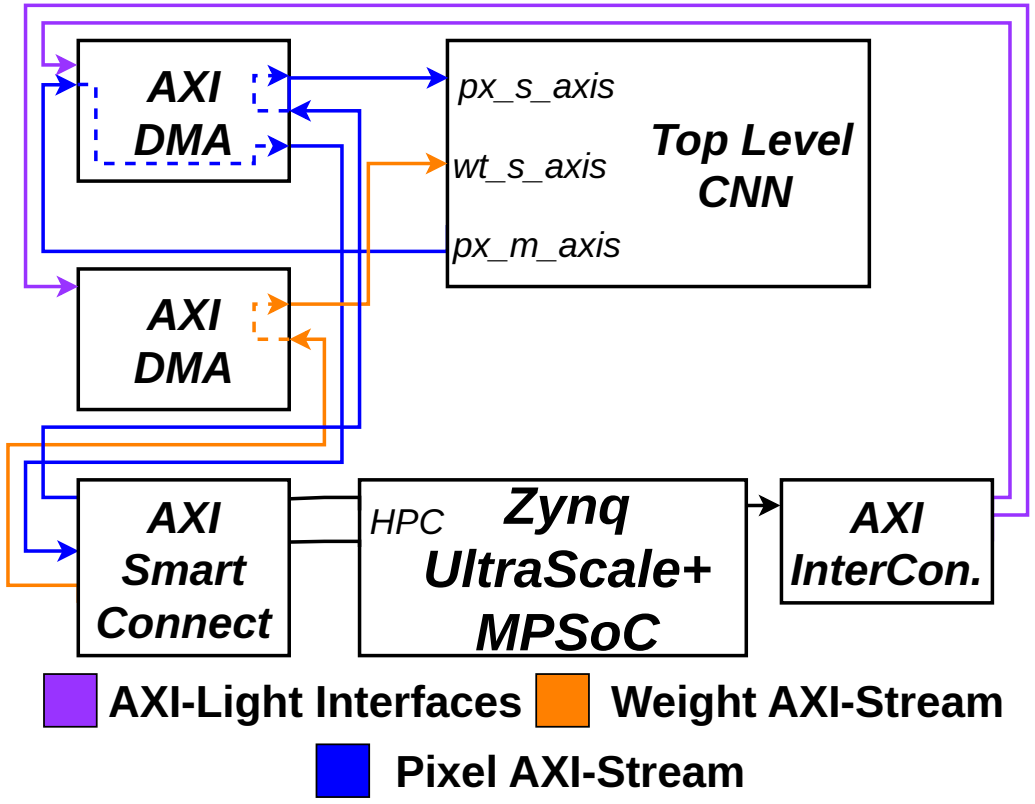
- Accumulation via circular ring buffer

⇒ Depth of FIFO set during reset to $image_width^2$

- Only instantiated once and used for Convolution Layers only
- Handles shifting needed for quantization

Block Design

Integration of the Design with the PS



- Two **Xilinx DMA** IP cores for the pixel and weight data

$\Rightarrow$ DMA core for weight data only with read channel

- **Bus Width:** 128 Bits (maximize throughput)
- **Burst Size:** 16 (Zynq UltraScale+ MPSoC is AXI-3 compliant)
- Arbitration for data lines using **Xilinx SmartConnet** core
- Arbitration for control lines using **Xilinx AXI Interconnect** core

7. Related Work

8. Tooling

9. Further Implementation

10. Further Evaluation

Hierarchical Resource Consumption

Complete Hierarchy of 3 × 3 Convolution Accelerator

Instance (Accelerator)	LUTs	FFs	BRAM
Top_Level_wrapper	19989 (28.33%)	23126 (16.39%)	38.5 (17.84%)
top_level_cnn_wrapper_0	12205 (17.30%)	11198 (7.94%)	31 (14.15%)
(top_level_cnn_wrapper_0)	792 (1.12%)	0 (0.00%)	0 (0.00%)
U0	11419 (16.18%)	11198 (7.94%)	31 (14.15%)
top_level_cnn_inst	11419 (16.18%)	11198 (7.94%)	31 (14.15%)
(top_level_cnn_inst)	0 (0.00%)	48 (0.03%)	0 (0.00%)
axis_2_configurable_slider_wrapper_inst	2801 (3.97%)	3031 (2.15%)	20 (9.26%)
(axis_2_configurable_slider_wrapper_inst)	0 (0.00%)	2 (0.01%)	0 (0.00%)
axis_2_pxs_inst	854 (1.21%)	1055 (0.75%)	8 (3.70%)
configurable_slider_inst	1948 (2.76%)	1974 (1.40%)	12 (5.56%)
control_unit_axis_pipeline_step	326 (0.46%)	292 (0.21%)	0 (0.00%)
control_unit_inst	876 (1.24%)	1505 (1.07%)	0 (0.00%)
(control_unit_inst)	59 (0.08%)	43 (0.03%)	0 (0.00%)
config_unit_inst	263 (0.37%)	239 (0.17%)	0 (0.00%)
weight_unit_inst	555 (0.79%)	1223 (0.87%)	0 (0.00%)
pxs_2_axis_inst	937 (1.33%)	724 (0.51%)	8 (3.70%)
(pxs_2_axis_inst)	0 (0.00%)	21 (0.01%)	0 (0.00%)
axis_pipeline_step_inst	78 (0.11%)	174 (0.12%)	0 (0.00%)
bram_fifo_array_inst	844 (1.20%)	512 (0.36%)	8 (3.70%)
pxs_2_axis_demux_inst	15 (0.02%)	17 (0.01%)	0 (0.00%)
routed_kernel_cores_inst	6388 (9.05%)	5474 (3.88%)	0 (0.00%)
KERNEL_CORES_INST	6228 (8.83%)	4266 (3.02%)	0 (0.00%)
ROUTING_CORES_INST	160 (0.23%)	1208 (0.86%)	0 (0.00%)
temp_layer_accumulator_inst	120 (0.17%)	124 (0.09%)	3 (1.39%)

Instance (IP cores)	LUTs	FFs	BRAM
axi_dma_0	2036 (2.89%)	3147 (2.23%)	5 (2.31%)
axi_dma_1	639 (0.91%)	961 (0.68%)	2.5 (1.16%)
axi_smc	3784 (5.36%)	6340 (4.49%)	0 (0.00%)
ps8_0_axi_periph	1314 (1.86%)	1446 (1.02%)	0 (0.00%)
rst_ps8_0_99M	15 (0.02%)	34 (0.02%)	0 (0.00%)

Performance Evaluation

Scaling of Stride Length

